

MIVA OPEN UNIVERSITY

FACULTY OF COMPUTING

DEPARTMENT OF SOFTWARE ENGINEERING

**DESIGN AND IMPLEMENTATION OF AN AI-ENHANCED MULTI-TENANT
FAMILY FUND MANAGEMENT SYSTEM**

BY

AMINU DANLADI HUSSAIN

2024/A/SENG/0156

**A PROJECT SUBMITTED TO THE DEPARTMENT OF SOFTWARE
ENGINEERING, FACULTY OF COMPUTING, MIVA OPEN UNIVERSITY, IN
PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF THE
DEGREE OF BACHELOR OF SCIENCE (B.Sc.) IN SOFTWARE ENGINEERING**

SUPERVISOR: DR. AYODEJI SAMUEL MAKINDE

SEPTEMBER, 2026

DECLARATION

I, Aminu Danladi Hussain (Matriculation Number: 2024/A/SENG/0156), declare that this report accurately presents the design, implementation, testing, and limitations of the project titled “Design and Implementation of an AI-Enhanced Multi-Tenant Family Fund Management System.” All sources consulted have been acknowledged in the reference list, and no data, test result, model metric, or system evidence has been knowingly fabricated. Forms of technical and editorial assistance, including the limited use of generative artificial intelligence, are disclosed in Section 5.9. This report has not been submitted elsewhere for the award of another degree.

Aminu Danladi Hussain

Student

Signature

Date

CERTIFICATION

This is to certify that this project titled "Design and Implementation of an AI-Enhanced Multi-Tenant Family Fund Management System" was carried out by Aminu Danladi Hussain (Matriculation Number: 2024/A/SENG/0156) of the Department of Software Engineering, Faculty of Computing, Miva Open University.

Dr. Ayodeji Samuel Makinde

Supervisor

Signature

Date

Head of Department

Signature

Date

External Examiner

Signature

Date

DEDICATION

This project is dedicated to my family, whose collective spirit of contribution, mutual support, and togetherness inspired the problem addressed in this study.

ACKNOWLEDGMENTS

I express my sincere gratitude to my supervisor, Dr. Ayodeji Samuel Makinde, for his guidance, constructive feedback, and encouragement throughout this project. I also appreciate the Department of Software Engineering, Miva Open University, for providing the academic environment in which the study was undertaken.

I am grateful to my family, whose practical experience of managing shared contributions inspired the problem addressed by this project. My appreciation also goes to my classmates and friends for their moral support and useful technical discussions, and to the open-source community, particularly the maintainers of Laravel, Vue.js, and the supporting libraries used in developing the system.

Above all, I am grateful to Almighty God for the wisdom, strength, and perseverance to complete this work.

ABSTRACT

Informal family contribution funds often depend on notebooks, spreadsheets and messaging platforms, making it difficult to maintain consistent balances, allocate partial payments and review financial decisions. This study designed and implemented FamilyFunds, an AI-enhanced multi-tenant web application for administering shared family funds. An iterative software development approach informed requirements analysis, system design, implementation and automated testing. The application uses Laravel, Vue.js with Inertia.js and PostgreSQL. Each family operates within a logically isolated workspace, with membership-based permissions for administrators, financial secretaries and members. The system supports monthly contributions, manual payments, Paystack integration, expenses, adjustments, reminders and financial reports. An oldest-balance-first algorithm allocates partial and lump-sum payments consistently, while separate receipt and allocation records support traceability. Reconciliation was added during implementation as an audit-focused extension. AI enhancement consists of a permission-controlled assistant and narrative report-summary integration; financial calculations and authorisation remain within the application. Automated tests verified the access and confirmation controls, but live-provider response quality was not evaluated. The local verification run recorded on 15 August 2026 passed 1,421 Pest tests with 6,904 assertions and 100.0% PHP code coverage. Twenty redacted screenshots document workflows using synthetic demonstration records. These results support the implemented software paths rather than predictive accuracy or production readiness. A supplementary logistic-regression prototype was also developed, but no authorised historical dataset was available for training or held-out evaluation. Predictive analytics therefore remains future work, and no model accuracy is claimed. Hosted verification, live integrations, performance measurement and recovery testing remain outstanding. The study demonstrates how family-specific roles, consistent payment allocation and reviewable records can be combined in one application, while identifying the validation required before wider operational use.

Keywords: Family Fund Management, Multi-Tenancy, Role-Based Access Control, Payment Allocation, AI Assistance, Financial Reporting

TABLE OF CONTENTS

Preliminary Pages.....	i
Title Page.....	i
Declaration.....	ii
Certification.....	iii
Dedication.....	iv
Acknowledgments.....	v
Abstract.....	vi
Table of Contents.....	vii
List of Tables.....	ix
List of Figures.....	x
List of Listings.....	xi
List of Abbreviations.....	xii
Chapter One: Introduction.....	1
1.1 Background to the Study.....	1
1.2 Statement of the Problem.....	2
1.3 Aim of the Study.....	4
1.4 Objectives of the Study.....	4
1.5 Research Questions.....	5
1.7 Significance of the Study.....	5
1.8 Scope of the Study.....	6
1.9 Limitations of the Study.....	7
1.10 Definition of Terms.....	8
1.11 Organisation of the Report.....	9
Chapter Two: Literature Review.....	10
2.1 Introduction to the Chapter.....	10
2.2 Conceptual Review.....	10
2.3 Theoretical Framework.....	14
2.4 Empirical Review of Related Works.....	15
2.5 Review of Existing Systems/Tools.....	17
2.6 Comparative Analysis of Related Works.....	18
2.7 Identified Research Gap.....	19

2.8 Summary of the Chapter.....	20
Chapter Three: Methodology.....	21
3.1 Introduction to the Chapter.....	21
3.2 Research Design / Project Approach.....	21
3.3 Analysis of Existing System.....	22
3.4 Proposed System Overview.....	23
3.5 System Requirements.....	25
3.6 Data Collection Methods.....	29
3.7 Population and Sampling.....	29
3.8 System Architecture / Design.....	30
3.9 Use Case / UML Diagrams.....	31
3.10 Database Design.....	35
3.11 Algorithm / Model Design.....	38
3.12 Tools and Technologies.....	39
3.13 Ethical Considerations.....	41
3.14 Summary of the Chapter.....	42
Chapter Four: System Implementation and Testing.....	43
4.1 Implementation of the System Design.....	43
4.2 Module Integration and Coding.....	48
4.3 Testing Strategy and Procedures.....	76
4.4 Test Results and Discussion.....	86
Chapter Five: Summary, Conclusion and Recommendations.....	93
5.1 Introduction.....	93
5.2 Summary of Findings.....	93
5.3 Achievement of Objectives.....	95
5.4 Contributions of the Study.....	97
5.5 Conclusion.....	98
5.6 Limitations.....	98
5.7 Recommendations.....	99
5.8 Future Work.....	100
5.9 AI-Assistance Disclosure.....	101
References.....	102

LIST OF TABLES

Table No.	Title	Page
1.1	Comparison of Existing Financial Management Platforms	3
1.2	Definition of Key Terms	8
2.1	Empirical Review of Related Works	15
2.2	Review of Existing Systems and Tools	18
2.3	Comparative Analysis of Related Works	18
3.1	Summary of Proposed System Features	24
3.2	Functional Requirements	25
3.3	Non-Functional Requirements	27
3.4	Deployment Requirements	28
3.5	Major Database Entities	36
3.6	Oldest-Balance-First Payment Allocation Algorithm	38
3.7	Tools and Technologies	39
4.1	Implementation and Evidence Environment	44
4.2	Design-to-Implementation Traceability	46
4.3	Chapter Four Interface Evidence Register	58
4.4	Testing Strategy and Evidence Sources	77
4.5	Structured System Test Cases and Current Results	79
4.6	Predictive Prototype Test and Evidence Matrix	83
4.7	Non-Functional Requirement Evaluation	86
4.8	Predictive Training and Evaluation Evidence Status	88
4.9	Objectives and Research Questions Verdict Matrix	91
5.1	Achievement of Study Objectives	95

LIST OF FIGURES

Figure No.	Title	Page
3.1	Existing Manual Family Fund Process	23
3.2	System Architecture Diagram	31
3.3	Use Case Diagram	32
3.4	Payment Allocation Flowchart	33
3.5	Entity-Relationship Diagram (ERD)	36
4.1a	Login Entry for the Synthetic Demonstration Tenant	59
4.1b	Password Confirmation Before a Sensitive Setting	59
4.1c	Account Security, Two-Factor and Passkey Settings	60
4.2	Family Administrator Dashboard	61
4.3	Member, Role and Contribution-Category Management	61
4.4	Contribution Register and Payment States	62
4.5	Oldest-First Payment Allocation Review	63
4.6a	Member Contribution Statement	64
4.6b	Receipt Showing Split Allocation Across Periods	65
4.6c	Paystack Self-Pay Review State Before Submission	66
4.7a	Expense Register	67
4.7b	Fund-Adjustment Register	67
4.8a	Reports, Exports and Schedule Controls	68
4.8b	Generated Financial Report Output	69
4.9	Seeded Contribution-Reminder Notification	70
4.10	Current Growth Plan and Feature Entitlements	71
4.11	Existing Controlled-AI Conversation in the Synthetic Tenant	72
4.12	Predictive Analytics Model-Unavailable State	73
4.13	Reconciliation Workspace and Seeded Settlement Candidate	74
4.14	Responsive Member Dashboard at 390 Pixels	75

LIST OF LISTINGS

Listing No.	Title	Page
4.1	Family Context Binding	49
4.2	Oldest-Balance-First Payment Allocation	50
4.3	Paystack Webhook Reference Verification	51
4.4	Confirm-First AI Write Governance	52
4.5	Logistic Regression Training Code for the Unevaluated Prototype	55
4.6	Planned Baseline Comparison and Activation Gates	56

Listings 4.5 and 4.6 show the implemented training and evaluation path. They do not represent an executed production training run or substitute for the missing authorised dataset and model evidence.

LIST OF ABBREVIATIONS

Abbreviation	Full Meaning
2FA	Two-Factor Authentication
AI	Artificial Intelligence
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
ERD	Entity-Relationship Diagram
HMAC	Hash-based Message Authentication Code
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
LLM	Large Language Model
ML	Machine Learning
MVC	Model-View-Controller
NGN	Nigerian Naira
ORM	Object-Relational Mapping
PHP	PHP: Hypertext Preprocessor
RBAC	Role-Based Access Control
REST	Representational State Transfer
SaaS	Software as a Service
SHA	Secure Hash Algorithm
SPA	Single Page Application
SQL	Structured Query Language
SSR	Server-Side Rendering
UI	User Interface
UX	User Experience
WebAuthn	Web Authentication

CHAPTER ONE

INTRODUCTION

1.1 Background to the Study

Digital finance has changed how people save, transfer money, and keep records, yet many collective financial practices still depend on informal administration. Across Sub-Saharan Africa, households and small groups continue to rely on cooperative saving, rotating contributions, mutual aid, and family-level support systems because these arrangements are close to the people who use them and are flexible enough to respond to emergencies. Recent studies on financial inclusion still show that informal savings and group-based financial arrangements remain relevant in Nigeria, especially where formal products are too expensive, too distant, or not trusted enough for everyday use (Demirgüç-Kunt et al., 2022; EFINA, 2023; Kouandou & Zeh, 2024).

Nigeria has a long history of collective finance. The Yoruba *ajo* and *esusu* systems, the Hausa *adashi*, and similar community-based arrangements are usually built on periodic contributions, social pressure, and shared expectations of fairness (Akanji, 2006; Iganiga & Asemota, 2008; Oloyede, 2008). Some of these systems are rotating, where members take turns receiving the pooled amount. Others are non-rotating and exist mainly as a common purse for emergencies, ceremonies, school fees, medical bills, property maintenance, or family welfare. This second arrangement is the focus of this project. It is referred to in this study as a family fund or family purse.

Although family funds are socially important, their administration is still mostly manual. A family may keep a notebook, a spreadsheet, a WhatsApp thread, or a mixture of all three. These tools are convenient at first, but they are weak as financial records. They do not enforce roles, they do not allocate partial payments consistently, they do not produce reliable audit trails, and they are difficult to review when disputes arise. A payment screenshot posted in a messaging group may prove that money was sent, but it does not show which month the payment covers, whether it clears an old balance, or whether the family treasurer has updated the record correctly.

At the same time, Nigeria's digital financial services market has grown rapidly. Payment gateways and savings platforms have made online transactions familiar to many users, while

the policy discussion around financial inclusion increasingly recognises digital channels as a route to wider access (Wezel & Ree, 2023). However, most popular platforms are designed either for individual savers or for formal lending and cooperative structures. PiggyVest and Cowrywise support personal saving and investment; CreditClan and Lendsqr focus more on formal lending infrastructure. None of these tools is designed primarily for informal family contribution funds with tiered member contributions, family-level roles, partial payments, expenses, reminders, and transparent reporting.

Modern software architecture makes it possible to address this gap. A multi-tenant web application can allow many families to use one platform while keeping each family's data logically separate. Role-based access control can distinguish the work of a family administrator, a financial secretary, and an ordinary member. Payment gateways such as Paystack can support online contributions, while PostgreSQL can provide structured relational storage for members, contributions, payments, expenses, and reports. In addition, recent advances in artificial intelligence make it possible to add controlled assistant features and plain-language report summaries, provided the system protects financial data and avoids unsupported claims in AI output (Brown et al., 2020; Kang & Liu, 2023).

This study therefore proposes FamilyFunds, an AI-enhanced multi-tenant family fund management system. The project is not merely a record book moved online. It is a structured system for managing contribution categories, generating monthly obligations, recording online and offline payments, allocating partial payments to the oldest outstanding balances first, tracking expenses and adjustments, producing reports, and giving authorised users secure access to the information they need. The work is timely because family funds remain socially useful, manual administration remains fragile, and the available digital tools still do not reflect the way many Nigerian families organise shared financial responsibility.

1.2 Statement of the Problem

Family contribution funds are often trusted because they are managed within a family, but that same informality creates administrative weaknesses. The person keeping the records may be honest and committed, yet mistakes can still occur when payments are late, partial, or made through different channels. A notebook can be misplaced. A spreadsheet can be overwritten. A WhatsApp message can be buried under hundreds of unrelated messages. Once the record is unclear, family members may disagree about who has paid, how much is outstanding, and whether expenses were properly authorised.

The problem is not simply that family funds lack software. The deeper problem is that existing software does not fit the structure of the practice. Individual savings applications do not support shared governance. Lending platforms are built for formal credit operations rather than informal family welfare. Generic spreadsheets allow calculations but do not enforce tenant isolation, member roles, invitations, payment verification, or audit-friendly records. As a result, families continue to manage money through tools that were never designed for the trust and accountability demands of a family purse.

Table 1.1 compares common digital finance tools with the requirements of the proposed system.

Table 1.1: Comparison of Existing Financial Management Platforms

Feature	PiggyVest	Cowrywise	CreditClan	Lendsqr	FamilyFunds
Main user focus	Individuals	Individuals	Cooperatives and lenders	Digital lenders	Family groups
Family workspace isolation	No	No	Limited	Limited	Yes
Tiered contribution categories	No	No	No	No	Yes
Admin, secretary, and member roles	No	No	Limited	Limited	Yes
Partial payment allocation	No	No	No	No	Yes
Expense and adjustment tracking	No	No	Limited	Limited	Yes
Online payment support	Yes	Yes	Yes	Yes	Yes
Family-readable reports	Limited	Limited	Limited	Limited	Yes
AI assistant or narrative summaries	No	No	No	No	Implemented integration; live response quality not evaluated
Predictive analytics	No	No	No	Yes	Unevaluated prototype; no active model

The specific problem addressed in this study is the lack of suitable digital support for managing informal family contribution funds securely and transparently. The tools reviewed do not bring together multi-tenant family separation, role-based governance, tiered contribution tracking, deterministic payment allocation, expense management, online payment support, and understandable reporting for this setting. This gap exposes families to inaccurate records, weak accountability, delayed follow-up, and avoidable disputes.

1.3 Aim of the Study

The aim of this study is to design and implement FamilyFunds, an AI-enhanced multi-tenant web application for managing family contribution funds with secure member access, structured contribution tracking, payment allocation, expense recording, reminders, and financial reporting.

1.4 Objectives of the Study

The objectives of the study are to:

1. Review the management practices, digital tools, and literature related to informal savings groups, family contribution funds, multi-tenant systems, access control, payments, and AI-supported reporting.
2. Design a multi-tenant family fund management system with logical data isolation, role-based access control, contribution categories, payment allocation rules, reporting, and secure user access.
3. Implement the proposed system using Laravel, Vue.js, Inertia.js, PostgreSQL, Paystack, Laravel Fortify, Laravel AI SDK, and related development tools.
4. Integrate AI assistant and report-summary features in a controlled way that respects role permissions and uses available family fund data.
5. Explore a logistic-regression extension for estimating whether a monthly contribution remains outstanding at its due date, specifying its data requirements, prototype implementation and evaluation protocol. Training and empirical validation are deferred until authorised historical data is available.
6. Test the system against key requirements, including tenant isolation, role enforcement, payment allocation correctness, report accuracy, authentication security, and usability.

Following the defence feedback, the title and scope were revised to emphasise the implemented fund-management system. “AI-enhanced” refers to the controlled assistant and report-summary integration. The original fifth objective included training and evaluating a predictor; that empirical objective was not achieved. Its revised wording identifies the completed preparatory work without treating the original objective as fully met. Predictive performance is outside the demonstrated results of this study.

1.5 Research Questions

The study is guided by the following research questions:

1. What design features are required for a web-based system to manage informal family contribution funds securely and transparently?
2. How can multi-tenant architecture and role-based access control be applied to protect the records of separate family groups on one platform?
3. How can partial and lump-sum payments be allocated consistently across outstanding contribution balances?
4. To what extent can AI assistant and report-summary features improve access to understandable family fund information without exposing unauthorised data?
5. What data and validation requirements would be needed to evaluate a future contribution-risk predictor? The original empirical question, concerning logistic-regression performance against simple baselines, remains unanswered.
6. How can the system be tested to confirm that it meets functional, security, reliability, and usability requirements?

1.7 Significance of the Study

This study is significant to Nigerian families that maintain contribution funds because it addresses a problem that is often treated as a private inconvenience rather than a software design challenge. A family fund may be informal, but the money involved is real and the social consequences of inaccurate records can be serious. FamilyFunds provides a clearer structure: administrators manage settings and members, financial secretaries record operational transactions, and members can view their own obligations and payment history. This can reduce disputes and improve confidence in the fund.

The project is also useful to software engineering practice. It demonstrates how a modern Laravel and Vue application can be designed as a multi-tenant financial record system with tenant scoping, policy-based permissions, Paystack payment handling, PostgreSQL relational storage, scheduled contribution generation, reminders, passkeys, two-factor authentication, and AI features. The implementation therefore provides a concrete case study for applying established software engineering principles to an under-served local problem.

Academically, the study contributes by connecting separate areas of literature: informal finance, SaaS multi-tenancy, access control, payment processing, financial reporting, and AI-assisted interfaces. The novelty of the work is not that any single technology is new, but that these technologies are brought together for the family fund context. That context matters because a family fund is neither an individual wallet nor a formal cooperative loan system. It has its own governance structure, payment patterns, trust boundaries, and reporting needs.

The project may also benefit small community groups, religious associations, class sets, and workplace welfare groups that manage shared contributions without formal accounting systems. Although the design is centred on families, the underlying model can support any small group that needs transparent contributions, role-aware access, and simple financial reports.

1.8 Scope of the Study

The study covers the design and implementation of a web-based family fund management system. Each family is treated as a tenant, meaning that its records are separated from other families' records. The functional scope includes family workspace setup, member invitation, contribution category management, monthly contribution generation, manual payment recording, Paystack payment initiation and verification, oldest-balance-first payment allocation, expense tracking, fund adjustments, reminders, subscription plans, dashboards, reports, AI assistant support, and AI-assisted report summaries.

The technical scope covers a Laravel 13 backend, Vue.js 3 frontend, Inertia.js 3 server-client bridge, PostgreSQL database, Tailwind CSS 4 styling, Laravel Fortify authentication, WebAuthn/passkeys, Paystack integration, Laravel AI SDK, Laravel Pennant feature flags, a Python 3.12/scikit-learn offline predictive package, Vite, Git, and Pest testing. The application is designed for web and mobile-browser access rather than native mobile deployment.

The study does not cover cryptocurrency payments, direct bank API integration, tax computation, full double-entry accounting, regulatory management for formal cooperative societies, or multilingual interfaces. Predictive analytics is an exploratory extension, documented as a software prototype and a protocol for future research. No model was trained or activated using genuine historical records, and no predictive performance is claimed. The planned comparison with training-prevalence and previous-period-late baselines remains future work. The AI assistant and report-summary functions are supporting features whose access controls were tested; their live response quality and usefulness were not empirically established.

1.9 Limitations of the Study

The first limitation is internet dependency. FamilyFunds is a web application, so members need a working device and internet access. This may affect users in areas with unstable connectivity.

The second limitation is payment-channel scope. Paystack is used as the online payment gateway because the project is designed around Nigerian use. Families outside Paystack-supported contexts would need other gateways.

The third limitation is data availability for prediction. No authorised historical training dataset was obtained, so the study could not train a model or measure predictive performance. Synthetic demonstration records were used to exercise application workflows, including full, partial and overdue payments. Those records do not establish how a model would perform on real family payment behaviour. Training and validation remain future work; the application can track payments and produce financial reports independently of this extension.

The fourth limitation concerns AI output. Large Language Models can produce fluent but inaccurate responses if they are not constrained (Kang & Liu, 2023). For this reason, AI features in FamilyFunds must be grounded in system data, limited by user permissions, and validated before they are trusted for financial decisions.

The fifth limitation is project scale. The system is designed for multiple families, but very large-scale performance testing across thousands of tenants is beyond the scope of this study.

1.10 Definition of Terms

Table 1.2: Definition of Key Terms

Term	Definition
Family fund	A non-rotating pool of money contributed by family members for shared needs such as welfare, emergencies, ceremonies, and family projects.
Tenant	A separate family workspace within the same application, with its own members, contribution records, payments, expenses, and reports.
Multi-tenancy	A software architecture where one application serves many tenants while keeping their data and operations logically separated.
Role-Based Access Control (RBAC)	An access-control model where permissions are assigned according to user roles such as Administrator, Financial Secretary, and Member.
Contribution category	A family-defined tier that determines how much a member is expected to contribute each month.
Partial payment	A payment that is less than the full amount owed for one or more contribution periods.
Oldest-balance-first allocation	The rule that applies a payment to the earliest outstanding contribution balance before newer balances.
Payment gateway	A service that processes online payments between users, banks, cards, and the application. Paystack is used in this project.
Large Language Model (LLM)	An AI model capable of generating and interpreting human language. In this project, it supports assistant responses and report summaries.
Predictive analytics	The use of historical data to estimate future outcomes, such as whether a member is likely to miss a contribution deadline.
WebAuthn/passkey	A browser-based authentication standard that uses public-key credentials, often linked to device biometrics or security keys.

1.11 Organisation of the Report

This report is organised into five chapters. Chapter One introduces the problem, aim, objectives, research questions, significance, scope, limitations, and key terms. Chapter Two reviews informal finance, multi-tenancy, access control, payments, AI-assisted reporting, predictive analytics, and related systems. Chapter Three explains the methodology and design: development approach, requirements, data collection, architecture, UML diagrams, database, algorithms, tools, and ethics. Chapter Four presents system implementation, testing, results, and validation. Chapter Five summarises the study, conclusions, contributions, and recommendations.

CHAPTER TWO

LITERATURE REVIEW

2.1 Introduction to the Chapter

This chapter reviews the literature and existing systems that inform the design of FamilyFunds. It is organised around five connected themes: informal family and group finance, multi-tenant software architecture, role-based access control, digital payments, and AI-supported reporting. Predictive analytics is reviewed as background for a possible future extension, not as an empirically demonstrated contribution. The chapter moves from concepts and theory to empirical studies, existing platforms, comparative analysis, and the specific research gap addressed by this project.

2.2 Conceptual Review

2.2.1 Informal Collective Finance and Family Contribution Funds

Informal collective finance refers to financial arrangements maintained outside formal banking structures, usually through social trust, repeated interaction, and community accountability. In Nigeria, these arrangements include *ajo*, *esusu*, *adashi*, cooperative savings, welfare funds, and family purses. Earlier studies established the importance of rotating savings and credit associations in African economies, but recent financial inclusion reports still show that informal saving and group finance remain relevant in Nigeria (Demirgüç-Kunt et al., 2022; EFINA, 2023; Kouandou & Zeh, 2024).

The family contribution fund differs from the better-known rotating model. In a rotating savings association, members contribute at fixed intervals and the pooled amount goes to one member per cycle. In a family fund, contributions are normally retained in a central pool and later used for family obligations. The purpose is not only saving; it is shared welfare. This difference affects the software design. A family fund needs a permanent balance, expense records, contribution categories, partial-payment handling, reports, and governance rules. A rotating savings application may not need all of these.

Trust is central to both arrangements. However, trust in a family fund should not mean the absence of verification. When payment records are scattered across notebooks, private spreadsheets, and messaging platforms, the group relies heavily on the memory and integrity

of the financial secretary. That arrangement may work for a small group over a short period, but it becomes weaker as membership grows, members migrate, contribution categories change, and payments arrive through different channels. The concept of a family fund therefore includes both social trust and the need for reliable records.

2.2.2 Multi-Tenant Software Architecture

Multi-tenancy is a software architecture in which one application instance serves multiple independent tenants. In a business SaaS application, a tenant might be a company. In FamilyFunds, the tenant is a family. The architectural value is that one platform can support many families without deploying a separate application for each one. The main risk is tenant isolation: one tenant's data must never be exposed to another tenant (Amazon Web Services, 2020; Bezemer & Zaidman, 2010; Sharma & Kaur, 2021).

There are different data-isolation strategies. A system may provide a separate database for each tenant, a separate schema for each tenant, or a shared schema with a tenant identifier on tenant-owned records. Separate databases offer stronger physical isolation but increase cost and maintenance. A shared-schema design is cheaper and easier to operate for many small tenants, but it requires strict application-level controls. For FamilyFunds, shared-schema multi-tenancy is appropriate because the expected tenants are small family groups, not large institutions. The design therefore depends on family_id scoping, middleware, policies, and careful query boundaries.

Recent SaaS literature continues to treat isolation and tenant-specific quality requirements as central design concerns. Jia et al. (2021) reviewed multi-tenancy in cloud platforms and emphasised the need to balance shared resources with tenant-specific requirements. Sharma and Kaur (2021) also treated tenant-aware storage and isolation as a key concern in multi-tenant applications. For a family fund platform, this concern becomes personal: a tenant-isolation failure would not merely be a technical bug; it would expose one family's private financial records to another family.

2.2.3 Role-Based Access Control and Accountability

Role-Based Access Control (RBAC) assigns permissions to roles rather than assigning them directly to individual users. The classical RBAC model remains important because it is simple, understandable, and suitable for organisations where duties are clearly separated

(Sandhu et al., 1996). In FamilyFunds, the main roles are Family Administrator, Financial Secretary, Member, and Platform Super Administrator.

The family fund setting makes RBAC more than a security feature. It is a governance structure. A Family Administrator should be able to manage members, settings, categories, and subscriptions. A Financial Secretary should be able to record payments, expenses, reminders, and reports. A Member should be able to view personal contribution history and pay online, but not alter financial records for others. The Platform Super Administrator should manage platform-level plans and oversight without becoming the treasurer for any one family.

Accountability also depends on audit-friendly records. If a financial secretary records a payment, the system should store who recorded it, when it was recorded, which member it relates to, and how it affected the contribution balance. RBAC therefore works together with data design. Permissions prevent unauthorised actions, while records make authorised actions reviewable.

2.2.4 Digital Payments and Payment Allocation

Digital payment systems reduce the friction of sending and receiving money. In African contexts, mobile money and payment gateways have expanded access to financial transactions and have influenced how households send, receive, and manage funds (Jack & Suri, 2014; Mbiti & Weil, 2016; Wezel & Ree, 2023). For Nigerian web applications, Paystack provides a practical payment gateway because it supports online payment flows, transaction references, webhooks, and developer-facing APIs.

However, accepting a payment is only part of the problem. A family fund also needs to decide how the payment should be applied to outstanding balances. If a member owes for January, February, and March but pays only enough for one and a half months, the system needs a clear rule. FamilyFunds uses an oldest-balance-first allocation rule. This is similar to a first-in, first-out treatment of obligations: older unpaid contributions are settled before newer ones. The benefit is consistency. The same payment history will always produce the same allocation result, which reduces disagreement and makes member statements easier to understand.

2.2.5 AI Assistance, Reporting, and Predictive Analytics

AI features in a financial system must be handled carefully. Large Language Models can help users ask questions in ordinary language and can turn structured report data into readable summaries. This is useful in a family fund because not every member is comfortable reading financial tables. Classical Natural Language Generation relied on templates and rule-based pipelines; modern LLMs are more flexible, but they also introduce the risk of fluent errors (Dale, 2020; de Wynter et al., 2023; Kang & Liu, 2023).

For this reason, the AI assistant in FamilyFunds is treated as a controlled interface, not as an independent financial authority. It should answer only within the user's family context, respect role permissions, use available system data, and avoid unsupported financial claims. Where actions are involved, the assistant should not silently change records without confirmation.

Predictive analytics is a separate but related concept. It uses historical contribution and payment data to estimate future behaviour, such as whether a contribution will remain outstanding at its due date. Credit-scoring literature provides evaluation ideas, including accuracy, precision, recall, F1-score, and attention to class imbalance (Hussin Adam Khatir & Bee, 2022; Robisco & Carbó Martínez, 2022). Logistic regression was selected for the exploratory FamilyFunds prototype because its coefficients and probabilities can be inspected. The proposed evaluation would compare it with a training-prevalence baseline and a previous-period-late baseline. However, findings from credit scoring cannot establish its suitability for small family funds. No authorised historical dataset was obtained in this study, so the selected method was not empirically evaluated and no predictive benefit is claimed.

2.2.6 Authentication, Usability, and Member Access

A family fund system must be secure, but it must also be usable by people with different levels of technical confidence. Strong authentication is necessary because the system stores personal and money-related records. At the same time, a login process that is too difficult may discourage older or less technical members from using the platform. WebAuthn and passkeys are relevant because they reduce dependence on passwords and use public-key authentication through browsers and devices (FIDO Alliance, 2022; W3C, 2021). Lyastani et al. (2020), however, showed that passwordless authentication still needs careful user support because people may not immediately understand the relationship between passkeys, biometrics, and account security.

The usability concept behind FamilyFunds is therefore role-sensitive simplicity. A member should not face the same interface as a financial secretary. A member mainly needs personal balance, payment history, payment action, and reminders. A financial secretary needs payment entry, expenses, reports, and follow-up tools. A family administrator needs members, categories, settings, and subscription controls. Good responsive interface design also matters because many users are likely to interact with the system on phones rather than desktop computers (Li et al., 2022). Security and usability are treated together because a system that users avoid cannot improve family accountability, no matter how strong its technical design is.

2.3 Theoretical Framework

This study is guided by an applied system-development and design-oriented framework. The project starts with a real problem, studies the context, designs an artefact, implements it, and evaluates whether it addresses the problem. This aligns with the practical logic of software engineering: the contribution is demonstrated through a working system, not only through abstract explanation.

The development approach is Agile and iterative. Agile is suitable because the project requirements are likely to evolve as the family fund workflow becomes clearer. A payment feature may reveal the need for allocation rules; allocation rules may affect reports; reports may affect the AI summary design. Research on Agile development shows that iterative work helps teams respond to changing requirements and manage quality concerns throughout development rather than treating them as late additions (Karhapää et al., 2021; Masood et al., 2020; Schwaber & Sutherland, 2020).

The architectural theory is a modular monolith supported by multi-tenancy. A modular monolith allows related modules to remain inside one deployable Laravel application while still separating responsibilities such as authentication, families, contributions, payments, reports, subscriptions, notifications, and AI. This is appropriate for a final-year system because the project is broad enough to need modularity but not large enough to justify separate microservices.

The access-control theory is RBAC. The RBAC model is used because the family fund domain already has role-like responsibilities. Instead of inventing complex permission rules, the system formalises familiar duties: administrators manage the family workspace, financial

secretaries manage money-related records, and members view and pay their own obligations. This theoretical choice supports both security and usability.

2.4 Empirical Review of Related Works

Empirical and technical studies related to this project can be grouped into informal finance, digital financial inclusion, multi-tenant systems, Agile development, AI reporting, and credit-risk prediction. Table 2.1 summarises selected works and their relevance to FamilyFunds.

Table 2.1: Empirical Review of Related Works

Author(s) and Year	Focus	Method/ Approach	Key Finding	Relevance to FamilyFunds
Adegbite and Machethe (2020)	Financial inclusion in Nigeria	Quantitative analysis	Gender and access gaps affect financial inclusion.	Shows that access to suitable financial tools remains uneven.
Demirgüç-Kunt et al. (2022)	Global financial inclusion	Global survey/databas e	Informal saving remains important in many economies.	Supports the need to consider informal finance in system design.
EFInA (2023)	Nigerian financial access	National survey	Formal inclusion improved, but informal services remain part of financial behaviour.	Provides recent Nigerian context for the problem.
Wezel and Ree (2023)	Digital financial services in Nigeria	IMF policy analysis	Digital services can support inclusion but need stronger uptake and trust.	Supports the use of digital channels while recognising adoption barriers.
Kouandou and Zeh (2024)	Informal savings in Nigeria	Household panel analysis	Informal savings help households manage shocks.	Reinforces the social value of informal savings structures.
Jia et al. (2021)	Multi-tenancy in cloud platforms	Systematic review	Tenant-specific quality and resource-sharing concerns are central.	Supports the system's focus on tenant isolation.
Sharma and Kaur (2021)	Tenant-centric SaaS storage	System design and evaluation	Tenant-aware storage improves isolation and efficiency.	Supports tenant-aware data design.
Karhapää et al. (2021)	Quality requirements in Agile	Multiple case study	Quality requirements must be managed throughout Agile work.	Justifies handling security, reliability, and usability from design stage.

Author(s) and Year	Focus	Method/ Approach	Key Finding	Relevance to FamilyFunds
Masood et al. (2020)	Agile team practices	Grounded theory study	Agile work benefits from self-organisation and iterative delivery.	Supports the iterative development approach.
Lyastani et al. (2020)	FIDO2 authentication	Comparative usability study	Passwordless authentication can improve usability but needs user understanding.	Supports passkeys with fallback authentication.
Dale (2020)	Natural language generation	State-of-the-art review	NLG moved from rule-based systems toward neural methods.	Supports AI-generated narrative reports.
de Wynter et al. (2023)	LLM output evaluation	Empirical evaluation	LLM output requires careful evaluation.	Supports validation of AI summaries.
Kang and Liu (2023)	LLM hallucination in finance	Empirical examination	Off-the-shelf LLMs can hallucinate in financial tasks.	Supports controlled, data-grounded AI assistant design.
Hussin Adam Khatir and Bee (2022)	Credit scoring	Comparative ML study	Model performance depends on algorithm and data-balancing choices.	Informs the evaluation and class-balance checks for logistic regression.
Robisco and Carbó Martínez (2022)	Credit default prediction	Model-risk evaluation	Predictive models should be evaluated beyond raw accuracy.	Supports using precision and recall for prediction.

The reviewed studies justify the main design decisions of the project. Informal savings research shows that informal finance is still relevant rather than obsolete. Digital financial inclusion studies show why online payments and accessible interfaces matter. Multi-tenancy studies show why tenant isolation must be explicit. Agile studies support iterative development. LLM studies warn that AI reports must be verified. Credit-scoring research provides metrics for logistic regression but also shows why a trained model must be compared with transparent baselines and must not be claimed without enough data.

One important pattern across these works is that technology alone does not solve a trust problem. Informal savings groups work partly because members know each other and can apply social pressure. A digital system should not remove that social layer; it should make the shared facts easier to verify. This is why FamilyFunds focuses on transparent balances, role-

aware actions, payment receipts, and reports. It supports the family structure rather than replacing it with a bank-like process.

2.5 Review of Existing Systems/Tools

Several existing systems are related to FamilyFunds, but none fully addresses the project problem.

PiggyVest is a Nigerian digital savings and investment platform for individuals. It supports personal savings goals and automated saving, which makes it useful for disciplined individual finance. Its limitation in this study is that it is not designed for family-level governance. It does not provide family tenants, financial-secretary roles, contribution categories, partial-payment allocation, or family expense reporting.

Cowrywise is also focused on individual saving and investment. It offers a polished digital experience and contributes to the broader adoption of financial technology in Nigeria. However, the platform is centred on personal financial goals, not informal family funds. It does not model a family purse with shared obligations, group reports, or role-separated record keeping.

CreditClan provides credit and lending infrastructure. It is closer to formal cooperative and lending workflows than to informal family welfare funds. Its strengths are credit-oriented features and payment support, but its assumptions differ from this project. A family fund does not require loan origination, credit underwriting, or formal lending operations as its core workflow.

Lendsqr provides lending infrastructure for digital lenders and financial institutions. It supports more advanced lending operations than a family fund needs. Its existence shows that Nigerian fintech has strong tools for credit businesses, but it also highlights the gap: the informal family contribution domain remains outside the main focus of such systems.

WhatsApp and spreadsheets are not financial platforms, but they are widely used in practice. They are accessible and familiar, which explains their popularity. Their weakness is that they are not structured record systems. They do not enforce roles, calculate balances reliably across partial payments, prevent accidental edits, verify online payments, or produce consistent family reports.

Table 2.2: Review of Existing Systems and Tools

System/Tool	Main Strength	Main Limitation for Family Funds
PiggyVest	Strong individual savings experience	No family workspace, RBAC, or contribution allocation
Cowrywise	Personal saving and investment support	Not designed for shared family welfare funds
CreditClan	Credit and cooperative finance infrastructure	More suited to formal lending than informal family funds
Lendsqr	Lending infrastructure and credit workflows	Focuses on lenders, not family contribution governance
WhatsApp	Familiar communication channel	No structured financial records, permissions, or reports
Spreadsheets	Flexible calculation and tabulation	Weak audit trail, weak access control, and error-prone collaboration

2.6 Comparative Analysis of Related Works

The comparison in Table 2.3 shows that related work and existing systems address parts of the problem but not the complete FamilyFunds requirement set.

Table 2.3: Comparative Analysis of Related Works

Area Compared	Prior Work or Tool	What It Covers	Gap Remaining
Informal finance	ROSCA and financial inclusion studies	Social value of informal savings and group finance	Limited attention to non-rotating family purses and software requirements
Digital financial inclusion	World Bank, EFINA, IMF studies	Importance of digital access and financial inclusion	Does not provide a system for family fund management
Individual savings platforms	PiggyVest, Cowrywise	Personal saving, investment, payment convenience	No family governance, tenant model, or shared fund reporting
Formal lending platforms	CreditClan, Lendsqr	Credit operations and lending infrastructure	Not designed for informal family contribution funds
Multi-tenant architecture	SaaS literature and cloud studies	Tenant isolation, resource sharing, scalability	Usually assumes business tenants, not family groups

Area Compared	Prior Work or Tool	What It Covers	Gap Remaining
RBAC	Access-control literature	Role-permission separation	Needs adaptation to family admin, secretary, and member roles
Digital payments	Paystack and mobile money literature	Online transactions and payment verification	Does not solve allocation across family contribution balances
AI reporting	NLG and LLM studies	Automated text generation and reporting support	Financial hallucination risk requires controlled, data-grounded summaries
Predictive analytics	Credit-scoring studies	Logistic regression, prediction metrics, and model-risk lessons	Family funds have small datasets and social defaults, so activation requires evidence beyond simple baselines

The key observation is that FamilyFunds is not trying to replace all existing tools. Instead, it combines the relevant strengths of several areas: the accessibility of digital finance, the separation of SaaS multi-tenancy, the accountability of RBAC, the convenience of online payment gateways, the consistency of payment-allocation algorithms, and the readability of AI-supported reporting. The gap lies in applying these ideas together to the informal family fund context.

2.7 Identified Research Gap

The literature and system review reveal four main gaps.

First, family contribution funds are under-represented in software-focused research. Informal finance literature often discusses rotating savings groups and financial inclusion, but the non-rotating family purse has different requirements. It needs contribution balances, shared expenses, adjustments, audit trails, and long-term reporting.

Second, existing digital finance platforms do not match family fund governance. Individual savings tools are useful for personal goals, and lending platforms are useful for formal credit operations, but neither category provides a role-governed family treasury with contribution categories and partial-payment allocation.

Third, multi-tenancy and RBAC have not been sufficiently discussed in the context of informal family finance. The technical pattern is established, but its application to family

trust boundaries is specific. The system must prevent cross-family data exposure while also enforcing role differences within each family.

Fourth, AI-supported reporting requires careful adaptation. LLMs may help explain financial tables, but generated text can contain unsupported statements. FamilyFunds implements a controlled assistant layer that uses permitted application records and requires confirmation for writes. Live response quality remains untested. The predictive literature also identifies a possible future research direction, but this study does not establish that logistic regression improves family-fund decisions.

The identified gap is the limited attention given to combining family-level governance, isolated workspaces, contribution tracking, deterministic payment allocation, expenses and understandable reporting in one system for informal family funds. Controlled AI assistance is a supporting interface within that system. Predictive modelling remains a separate opportunity for future investigation.

2.8 Summary of the Chapter

This chapter reviewed the concepts, theories, empirical studies, and existing tools relevant to FamilyFunds. It showed that informal finance remains important in Nigeria, but family contribution funds are still managed with weak manual tools. It also showed that multi-tenant architecture, RBAC, payment gateways, AI-assisted reporting, and predictive analytics each offer useful ideas, but none of them alone solves the family fund problem.

The review established the academic and practical justification for the proposed system. FamilyFunds is positioned as an engineering response to a specific gap: informal family funds need a secure, role-governed, multi-tenant, payment-aware, report-friendly system. Chapter Three builds on this literature by explaining the methodology, design decisions, requirements, diagrams, database structure, tools, and ethical considerations used to develop the proposed system.

CHAPTER THREE

METHODOLOGY

3.1 Introduction to the Chapter

This chapter explains the methodology used to design and develop FamilyFunds. The aim stated in Chapter One is to build an AI-enhanced multi-tenant web application for managing family contribution funds with secure access, contribution tracking, payment allocation, expense recording, reminders, and reporting. The objectives cover related work, system design, implementation, controlled AI assistance and software testing. An exploratory logistic-regression prototype and its proposed evaluation protocol are also documented. They describe preparatory work for future prediction research; no historical-data training or model evaluation was completed.

The chapter therefore covers the project approach, existing-system analysis, proposed-system overview, requirements, data collection, population and sampling considerations, architecture, UML and system diagrams, database design, algorithm and model design, tools and technologies, and ethical considerations. Each methodological choice is linked to the problem: family funds need reliable records, clear responsibilities, private family workspaces, accurate payment handling, and reports that members can understand.

3.2 Research Design / Project Approach

The study uses an applied system-development design. The output of the research is a working software artefact rather than only a survey or theoretical model. This approach is suitable because the central problem is practical: families need a better way to manage contribution funds than notebooks, spreadsheets, and scattered messages. The system is therefore designed, implemented, and validated as a response to identified requirements.

The software development approach is Agile and iterative. A strict Waterfall model was not selected because family fund requirements are likely to become clearer during design and implementation. A simple feature such as payment recording affects member balances, reports, reminders, AI summaries, and audit trails. If the system were designed once and frozen too early, important realities of the domain would be missed. Agile development

supports incremental delivery, review, and correction while still keeping the work tied to the objectives of the study (Karhapää et al., 2021; Schwaber & Sutherland, 2020).

The work followed five broad stages:

1. Problem and literature analysis, covering informal finance, existing platforms, multi-tenancy, RBAC, digital payments, AI reporting, and predictive analytics.
2. Requirements identification, based on the problems observed in manual family fund administration and the gaps established in Chapter Two.
3. System design, including architecture, data design, user roles, process flows, interface structure, payment allocation, and security controls.
4. Incremental implementation, where authentication, family management, contributions, payments, expenses, reports, reminders, subscriptions, AI features, and the predictive pipeline were developed in modules.
5. Validation planning, where the critical behaviours were mapped to tests and acceptance checks, especially tenant isolation, role enforcement, payment allocation, report accuracy, AI response control, and predictive data/evaluation gates.

AI assistant and report-summary integration were implemented as supporting application features, with live response evaluation left outstanding. The exploratory predictive package contains logistic-regression training code and two baseline methods, but its tests use controlled fixtures. The study did not obtain the authorised historical dataset needed to fit and evaluate a model for practical use.

3.3 Analysis of Existing System

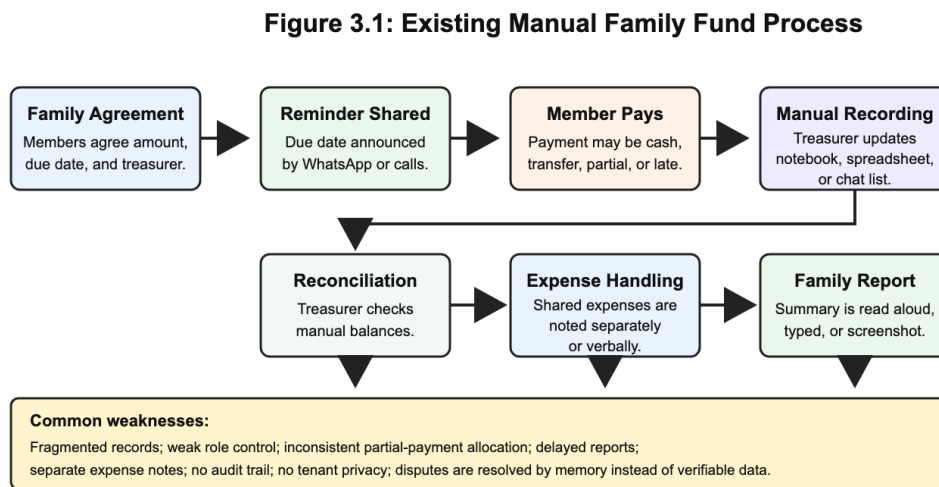
The existing system is the manual or semi-digital method commonly used by families to manage contribution funds. In this arrangement, a family appoints a treasurer or financial secretary. Members send contributions through cash, bank transfer, or informal handover. The financial secretary records payments in a notebook, a spreadsheet, a phone note, or a WhatsApp message thread. Reports are given verbally, posted as text summaries, or shared as screenshots.

This existing process has several weaknesses. First, records are fragmented. A payment may exist in a bank alert, a WhatsApp screenshot, and a handwritten note, but those pieces may not agree. Second, roles are informal. A family may know who the financial secretary is, but

the tools being used do not technically prevent other people from editing a spreadsheet or forwarding wrong information. Third, partial and lump-sum payments are difficult to allocate consistently. Fourth, expense records are often separate from contribution records, making it hard to calculate the true fund balance. Fifth, reporting depends on manual effort, so reports may be delayed, incomplete, or written in a way that ordinary members cannot interpret.

Figure 3.1 summarises the existing manual family fund process.

Figure 3.1: Existing Manual Family Fund Process



The major bottleneck in the existing system is that the family record depends too much on one person's manual discipline. Even when the person is honest, the process does not provide enough structure for verification, audit, payment allocation, or tenant-level privacy. These weaknesses justify the need for a purpose-built system.

3.4 Proposed System Overview

The proposed system is FamilyFunds, a multi-tenant web application for family contribution fund management. Each family operates in its own workspace, while the platform uses shared infrastructure. Users are assigned roles that determine what they can see and do. The system records members, contribution categories, monthly obligations, payments, expenses, adjustments, reminders, reports, subscription plans, and AI assistant interactions.

FamilyFunds resolves the problems in the existing system by moving the authoritative record into a structured PostgreSQL database. It also introduces role-based access control, Paystack

payment support, deterministic oldest-balance-first payment allocation, automated contribution generation, reminders, and clear reports. AI assistance is added as a controlled interface for answering permitted questions and summarising reports in plain language.

Table 3.1: Summary of Proposed System Features

Feature Area	Description
Multi-tenant family workspaces	Each family has a logically isolated workspace within the same application.
Role-based access control	Family Administrator, Financial Secretary, Member, and Platform Super Administrator have separate permissions.
Contribution categories	Families define monthly contribution amounts for groups such as employed members, students, or other categories.
Monthly contribution generation	The system creates contribution obligations for paying members based on category and due date.
Payment recording	Authorised users record offline payments, while members can initiate online payments through Paystack.
Payment allocation	Payments are applied to the oldest outstanding balances first.
Expenses and adjustments	Family spending, donations, corrections, and other balance movements are recorded.
Reminders	Email, browser push, and WhatsApp-related reminder flows support follow-up.
Reports	Monthly and annual reports show collections, balances, expenses, and member status.
AI assistant and summaries	The assistant answers role-permitted questions and supports plain-language report summaries.
Exploratory predictive prototype	Training and scoring code with activation controls; historical-data training, predictive evaluation and practical use are deferred.
Security controls	Email verification, password hashing, two-factor authentication, passkeys, signed invitations, policies, and tenant scoping protect the system.

Bank reconciliation was not part of the original requirement set above. It emerged during later Agile iterations when immutable receipts, expenses, adjustments, provider settlements, and imported bank transactions needed a governed way to be compared. Chapter Four therefore reports reconciliation as a post-design extension rather than presenting it retrospectively as an original requirement.

3.5 System Requirements

System requirements define what the system must do and the qualities it must maintain while doing so. The requirements below are written to be testable because vague requirements such as "the system should be fast" do not help implementation or validation.

3.5.1 Functional Requirements

Table 3.2: Functional Requirements

ID	Requirement	Description	Main Actor
FR1	User authentication	Users shall register, log in, verify email, reset passwords, and manage account security.	All users
FR2	Family workspace creation	A family administrator shall create and configure a family workspace.	Family Admin
FR3	Member invitation	The system shall allow administrators to invite members using secure invitation links.	Family Admin
FR4	Role assignment	The system shall support Family Admin, Financial Secretary, Member, and Platform Super Admin roles.	Family Admin
FR5	Contribution category setup	Administrators shall define contribution categories and monthly amounts.	Family Admin
FR6	Monthly contribution generation	The system shall generate monthly obligations for paying members.	Family Admin, System
FR7	Contribution status tracking	The system shall show unpaid, partially paid, paid, and overdue statuses.	All authorised users
FR8	Manual payment recording	Authorised officers shall record cash, transfer, or offline payments.	Family Admin, Financial Secretary

ID	Requirement	Description	Main Actor
FR9	Online payment processing	Members shall initiate online payments through Paystack where enabled.	Member
FR10	Payment verification	The system shall verify Paystack transactions before marking them successful.	System
FR11	Payment allocation	The system shall allocate payments to the oldest outstanding contribution balances first.	System
FR12	Expense recording	Authorised users shall record family expenses with amount, date, description, and recorder.	Family Admin, Financial Secretary
FR13	Fund adjustment recording	Authorised users shall record donations, corrections, and other non-contribution balance movements.	Family Admin, Financial Secretary
FR14	Reports	The system shall generate monthly and annual reports for contributions, payments, expenses, adjustments, and balances.	Family Admin, Financial Secretary
FR15	Member dashboard	Members shall view personal balances, payment history, and contribution status.	Member
FR16	Reminders	Authorised users shall send or trigger contribution reminders through supported channels.	Family Admin, Financial Secretary
FR17	Subscription management	Families shall access platform plans with member limits and feature availability.	Family Admin
FR18	AI assistant	The assistant shall answer questions using permitted family data and role-aware tools.	Authorised users
FR19	AI report summary	The system shall support plain-language summaries of financial reports.	Family Admin, Financial Secretary

ID	Requirement	Description	Main Actor
FR20	Predictive extension, deferred	The original requirement envisaged advisory due-date-outstanding estimates from a validated model. Only the software prototype and access controls were implemented; without authorised training and evaluation data, the interface must report prediction as unavailable.	Family Admin, Financial Secretary

FR20 is retained to trace the original design and its incomplete outcome. It is not counted as a delivered predictive service. NFR13 and NFR14 below specify safeguards for this deferred extension; software tests of these safeguards do not establish predictive quality.

3.5.2 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

ID	Category	Requirement	Expected Measure
NFR1	Security	A user shall access only records belonging to the user's family tenant.	Cross-family access attempts are denied.
NFR2	Security	Sensitive actions shall be restricted by role.	Members cannot record payments or alter family settings.
NFR3	Security	User accounts shall support strong authentication controls.	Email verification, 2FA, passkeys, and password hashing are available.
NFR4	Reliability	Payment allocation shall be deterministic.	The same payment history produces the same balances.
NFR5	Reliability	Paystack callbacks shall not create duplicate successful payments.	Repeated callback events do not double-record payments.
NFR6	Usability	Common workflows shall be usable on mobile and desktop browsers.	Key pages remain responsive and readable.
NFR7	Usability	Reports shall be understandable to non-accounting users.	Summary text and tables use clear labels and plain language.
NFR8	Performance	Dashboard and report pages shall remain responsive for ordinary family sizes.	Common pages load within an acceptable time during testing.
NFR9	Maintainability	Modules shall remain separated by responsibility.	Authentication, payments, reports, and AI logic are not mixed into one layer.

ID	Category	Requirement	Expected Measure
NFR10	Auditability	Financial records shall store relevant dates, amounts, users, and relationships.	Transactions can be reviewed and reconciled.
NFR11	Privacy	AI features shall use only the data needed for the user's request.	Prompts and tool calls avoid unnecessary personal exposure.
NFR12	Availability	The deployed system shall support HTTPS, queues, scheduler, and backups.	Required services are configured in deployment.
NFR13	Model governance	Predictive output shall be reproducible and activation-gated.	Dataset schema, feature version, split, coefficients, baselines, and evaluation artefacts are recorded before activation.
NFR14	Model safety	Predictive output shall be advisory, role-restricted, and honest about insufficient history.	Predictions do not trigger automatic penalties and unavailable cases show a clear reason.

3.5.3 Deployment Requirements

Table 3.4: Deployment Requirements

Requirement Type	Requirement
Runtime	PHP 8.4 or later with required Laravel extensions
Dependency tools	Composer, Node.js, npm, and Vite build process
Database	PostgreSQL database with secure credentials and backups
Web server	HTTPS-enabled Nginx, Apache, Laravel Cloud, or equivalent platform
Background processing	Queue worker and Laravel scheduler for jobs and reminders
Environment configuration	.env variables for database, mail, Paystack, AI provider, WhatsApp-related settings, and app keys
Security	HTTPS, password hashing, signed URLs, webhook validation, and restricted credentials
Optional future predictive runtime	Offline training tools and native Laravel scoring code; no trained model is included in the demonstrated system

3.6 Data Collection Methods

This project does not rely on a formal questionnaire or invented survey data. The data used for the methodology came from four honest sources. The first source was literature and document analysis, including studies on informal finance, digital financial inclusion, SaaS multi-tenancy, RBAC, digital payments, AI reporting, and predictive analytics. The second source was observation of typical family fund administration: how members are added, how contribution amounts are agreed, how payments are announced, and how balances are reconciled. The third source was analysis of existing tools such as PiggyVest, Cowrywise, CreditClan, Lendsqr, WhatsApp, and spreadsheets. The fourth source was the system's own development artefacts: routes, controllers, models, migrations, diagrams, requirements, and test scenarios.

Software testing and demonstration used synthetic family identities and financial records created by seeders or test factories. These records exercised full, partial and overdue payments and were not observations of actual members. The local demonstration ledger checked on 3 September 2026 contained 40 contribution obligations across eight monthly periods for five members; this count describes the demonstration ledger, not a training sample or the separate automated-test fixtures. No authorised historical dataset was obtained for predictive training. A future study would need a de-identified export with documented provenance, consent basis, period coverage and point-in-time correctness. The prototype's twelve-column export contract cannot reconstruct the time-varying effect of later reversals, so it requires histories containing reversed allocations to be excluded before the minimum-data checks. The specified minimums are 500 valid mature member-periods, 50 member histories, 12 chronological periods and 100 cases in each outcome class. These are proposed eligibility requirements, not counts achieved by the synthetic demonstration.

3.7 Population and Sampling

The study is mainly a system-development project, so it does not use a statistical population or probability sampling method. The relevant user population for design purposes consists of the people who normally participate in a family contribution fund: family administrators, financial secretaries, ordinary members, and the platform administrator.

Validation scenarios are selected purposively because the aim is to test whether the artefact behaves correctly in the most important use cases. The selected scenarios include creating a family, inviting members, assigning roles, generating contributions, recording partial

payments, initiating Paystack payments, viewing dashboards, sending reminders, generating reports, using AI assistance, preventing cross-family access, rejecting invalid predictive inputs, and showing unavailable or pooled predictive states when history is insufficient. This sampling approach is appropriate because correctness is judged by system behaviour under realistic workflows, not by generalising survey responses.

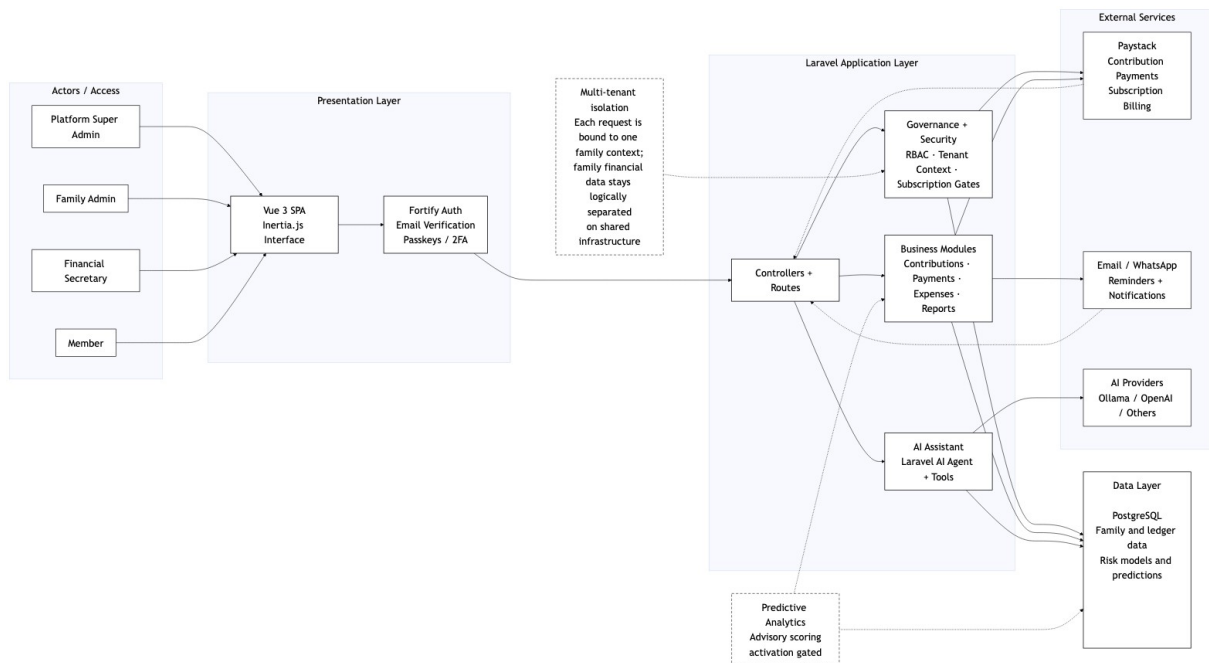
3.8 System Architecture / Design

FamilyFunds is designed as a layered, modular monolith. The frontend layer uses Vue.js and Inertia.js to present dashboards, forms, reports, settings, AI chat screens, and predictive advisory states. The application layer uses Laravel controllers, requests, services, policies, jobs, commands, and AI tools. The data layer uses PostgreSQL to store families, users, contributions, payments, expenses, adjustments, plans, notifications, passkeys, WhatsApp-related messages, AI conversations, payment-risk model metadata, and immutable predictions. A separate offline Python package validates authorised exports, trains and evaluates logistic regression, and produces a portable artefact for Laravel. External services include Paystack for payments and subscriptions, mail and messaging channels for communication, and AI providers for assistant functions.

Figure 3.2 shows the high-level architecture.

The predictive branch in the architecture represents the optional prototype. Its presence in the design does not mean that a model was trained, activated or used to produce the study's results.

Figure 3.2: System Architecture Diagram



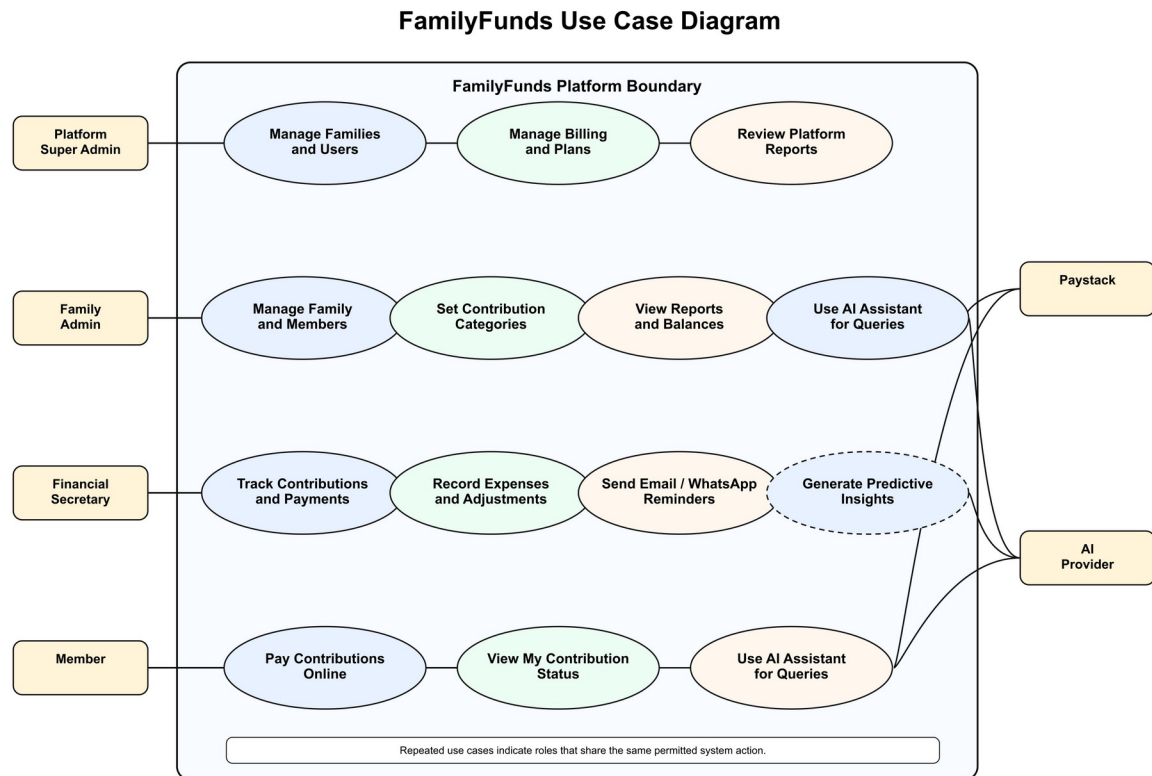
The architecture uses shared-schema multi-tenancy. Each family-specific record carries a `family_id` where appropriate. Middleware, policies, validation, and query scoping enforce the boundary. This strategy keeps the system practical for many small family groups while still maintaining logical separation. The design also keeps external services outside the authority of the application. Paystack verifies transactions, but Laravel decides how verified payments affect contribution records. The AI provider generates language, but Laravel controls permissions, tools, and data access.

3.9 Use Case / UML Diagrams

The system uses UML-style and process diagrams to explain the main actors, workflows, and data relationships. The diagrams are used because the system contains several roles, many related entities, and financial processes that are easier to verify visually than through prose alone.

Figure 3.3 presents the use case diagram.

Figure 3.3: Use Case Diagram



The main human actors are Platform Super Administrator, Family Administrator, Financial Secretary, and Member. The Platform Super Administrator manages platform-wide oversight. The Family Administrator manages the family workspace, members, settings, contribution categories, subscriptions, and reports. The Financial Secretary records payments, expenses, adjustments, reminders, and reports. Members view their own status and pay contributions. External actors include Paystack and AI providers, but they support rather than control the system.

The payment allocation process is one of the most important workflows because it affects member balances and family trust. Figure 3.4 presents the activity/flowchart view in readable panels.

Figure 3.4a: Payment Allocation Flowchart, Panel 1

Figure 3.4a: Payment Capture and Verification

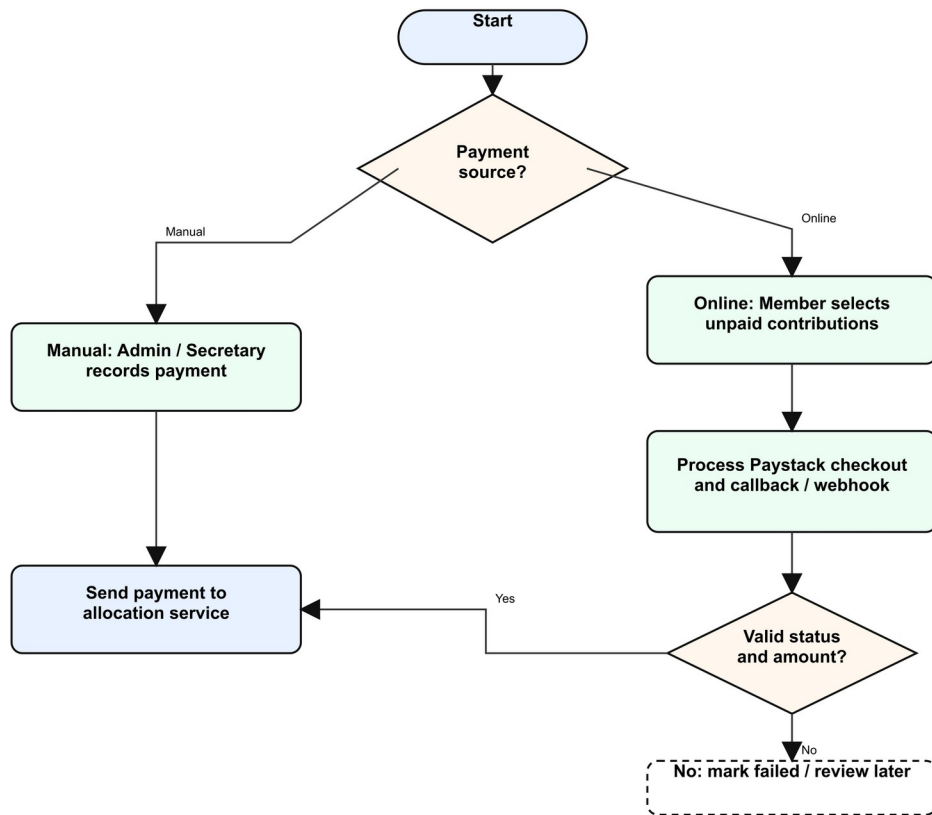


Figure 3.4b: Payment Allocation Flowchart, Panel 2

Figure 3.4b: Oldest-Balance-First Allocation

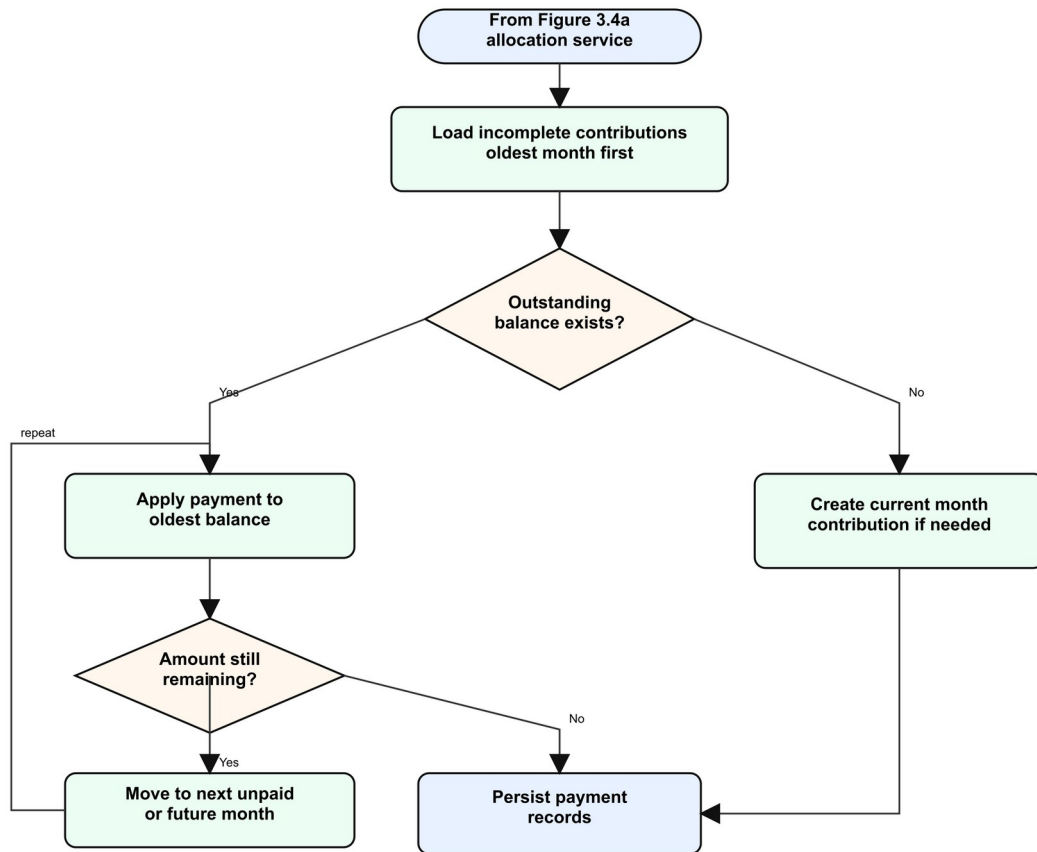
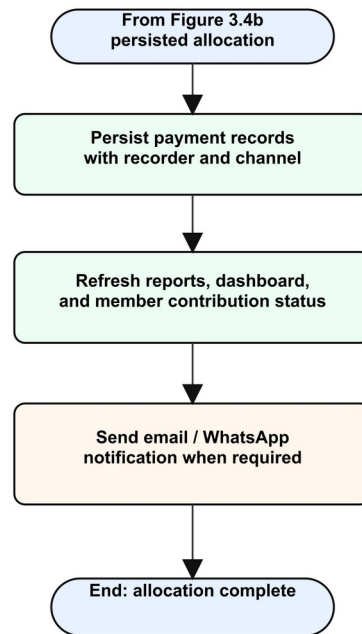


Figure 3.4c: Payment Allocation Flowchart, Panel 3

Figure 3.4c: Save Results, Update Records, Notify



The class-style model is represented by the main domain entities: Family, User, FamilyCategory, Contribution, Payment, Expense, FundAdjustment, PlatformPlan, PaystackTransaction, Passkey, Notification, WhatsAppMessage, AgentConversation, AgentConversationMessage, PaymentRiskModelVersion, and PaymentRiskPrediction. The sequence-style processes include invitation acceptance, contribution generation, manual payment recording, Paystack payment verification, reminder sending, report generation, AI assistant response generation, model installation, and advisory scoring.

3.10 Database Design

The database design uses PostgreSQL as the relational database. The design is centred on the family tenant. Tables that contain family-specific data include a family_id field or are connected through another family-owned entity. This supports shared-schema multi-tenancy while keeping relationships clear.

Figure 3.5 shows the entity-relationship diagram.

Figure 3.5: Entity-Relationship Diagram

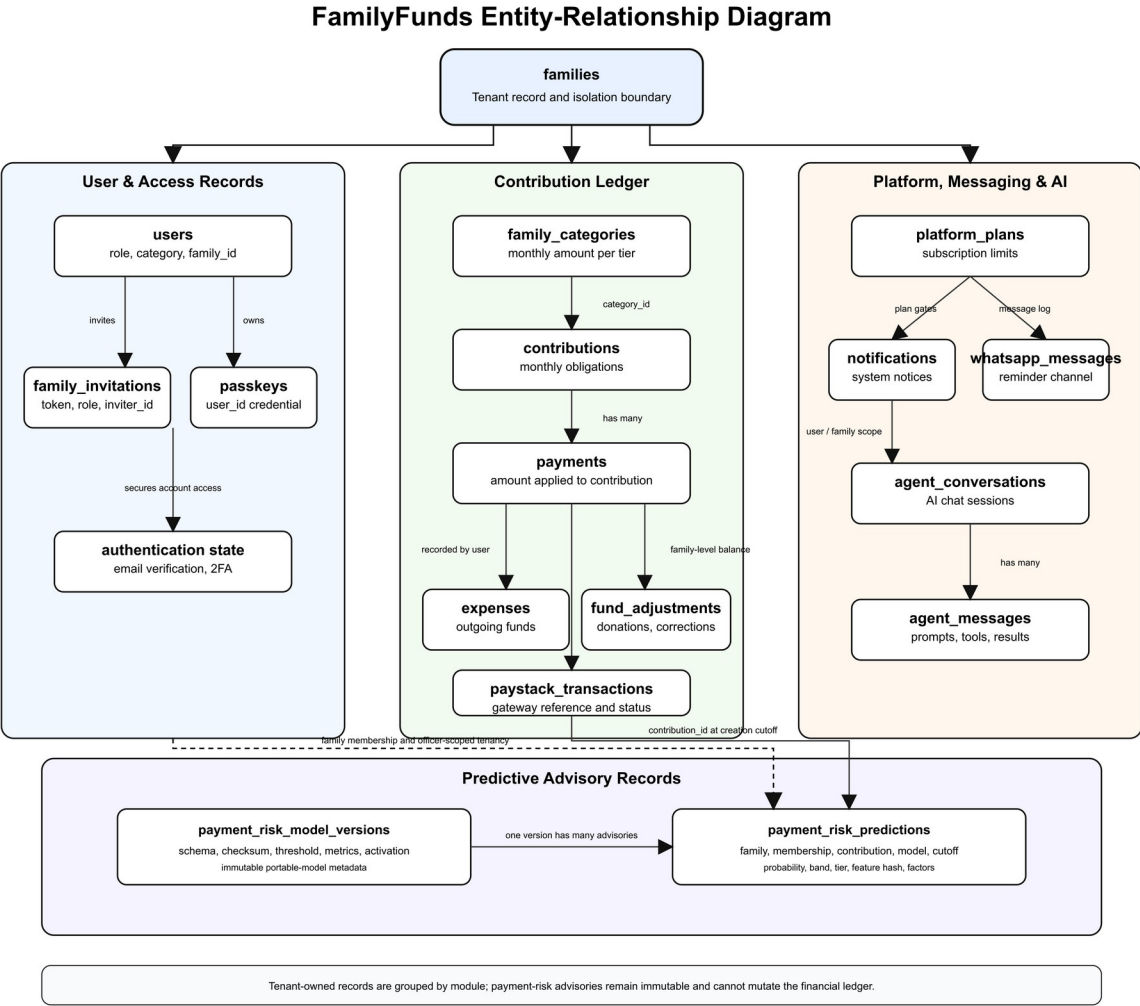


Table 3.5: Major Database Entities

Entity	Purpose	Key Relationship
families	Stores family workspace settings, currency, due day, status, billing, and bank details.	Has many users, categories, contributions, expenses, adjustments, invitations, and transactions.
users	Stores account, role, family membership, category, authentication, and profile data.	Belongs to a family and may belong to a family category.
family_categories	Stores contribution tiers and monthly amounts.	Belongs to a family and may be assigned to users.

Entity	Purpose	Key Relationship
family_invitations	Stores invitation email, role, token, expiry, and acceptance status.	Belongs to a family and inviter.
contributions	Stores monthly contribution obligations and statuses.	Belongs to a family and user; receives payments.
payments	Stores payment amounts applied to contribution obligations.	Belongs to a contribution and recorded-by user.
expenses	Stores family spending records.	Belongs to a family and recorded-by user.
fund_adjustments	Stores donations, corrections, and non-contribution balance changes.	Belongs to a family and recorded-by user.
paystack_transactions	Stores online payment and subscription transaction references.	Belongs to a family and user.
platform_plans	Stores subscription plan names, prices, limits, and features.	Used by families for subscription management.
passkeys	Stores WebAuthn credential information.	Belongs to a user.
notifications	Stores application notifications.	Linked to notifiable users.
whatsapp_messages	Stores inbound and outbound WhatsApp-related communication.	Linked to family and user where applicable.
agent_conversations	Stores AI assistant conversation sessions.	Belongs to a user.
agent_conversation_messages	Stores AI messages, tool calls, tool results, and metadata.	Belongs to an AI conversation.
payment_risk_model_versions	Stores immutable model schema, checksum, training window, threshold, dataset summary, metrics, and activation state.	Has many payment-risk predictions.
payment_risk_predictions	Stores immutable contribution-cutoff advisories, history tier, feature hash, model version, and factors.	Belongs to a family, membership, contribution, and model version.

The database design supports financial traceability by separating obligations from payments. A contribution records what a member owes for a period. A payment records money applied

to that obligation. This separation is important because one incoming amount can be split across more than one contribution.

3.11 Algorithm / Model Design

The main algorithm in the system is the oldest-balance-first payment allocation algorithm. It handles manual and verified online payments in a consistent way.

Table 3.6: Oldest-Balance-First Payment Allocation Algorithm

Step	Operation
1	Receive the member, payment amount, payment channel, and transaction details.
2	Retrieve the member's unpaid and partially paid contributions, ordered from oldest to newest.
3	For each contribution, calculate the outstanding balance.
4	Apply the available payment amount to the oldest outstanding balance.
5	Mark the contribution as paid if the balance reaches zero, or partially paid if a balance remains.
6	Continue until the payment amount is exhausted or all outstanding balances are cleared.
7	If an overpayment remains, apply it according to the system's advance-payment rules or record it for review.
8	Store payment records and update contribution statuses for reporting.

The AI assistant model is designed as a tool-aware assistant. The user's role and family context are checked before tools are used. Read-only questions, such as contribution summaries or fund balances, return information from permitted records. Write-related actions, such as recording expenses or payments through an assistant tool, require explicit confirmation and role permission.

The following predictive specification is retained for reproducibility and future development. It describes how the untrained prototype is intended to work; no historical-data fitting, threshold selection or held-out evaluation was carried out in this study.

The predictive component is a binary logistic regression classifier for the outcome "overdue" versus "on time," where overdue means that the system still recorded an outstanding balance at the end of the obligation's due date. The implementation uses fourteen point-in-time

features: expected amount, days available before the due date, calendar month and quarter, previous mature-period count, three-period, six-period and lifetime on-time rates, prior partial-payment rate, mean and median recorded settlement delay, prior outstanding count and amount, and overdue streak. Within the locked export, “prior outstanding amount” has the reproducible meaning of the due-date residual summed only for prior obligations that were still unsettled strictly before the target cutoff; later partial amounts cannot be reconstructed from the approved columns. Every feature must be computable strictly before the target obligation's creation-time cutoff; direct identity fields and future target-period payment information are excluded.

The approved evaluation is chronological: complete contribution periods are assigned 60% to training, 20% to validation, and 20% to a held-out future-like partition. Standardisation and an L2, class-balanced logistic regression are fitted on training data only. The threshold is selected on validation data by maximising overdue-class F1 subject to at least 0.70 recall, and the untouched partition is then used once for final evaluation. Logistic regression is compared with a training-prevalence baseline and a previous-period-late baseline. Accuracy is reported with balanced accuracy, overdue precision, recall and F1, ROC-AUC, PR-AUC, Brier score, bootstrap intervals, class distribution, and a confusion matrix because raw accuracy can be misleading when most members pay on time (Hussin Adam Khatir & Bee, 2022; Robisco & Carbó Martínez, 2022). Dataset schema, feature version, coefficients, intercept, decision threshold, and evaluation artefacts are versioned with the model. If the dataset lacks adequate history, both outcome classes, a valid held-out period, or improvement over the two baselines, training or activation fails closed and the interface reports that prediction is unavailable.

3.12 Tools and Technologies

The project uses tools selected for practicality, maintainability, and compatibility with the application requirements.

Table 3.7: Tools and Technologies

Category	Tool/Technology	Purpose
Backend language	PHP 8.4	Main server-side language.

Category	Tool/Technology	Purpose
Backend framework	Laravel 13	Routing, controllers, validation, queues, scheduler, policies, models, and services.
Frontend framework	Vue.js 3	User interface pages, dashboards, forms, and interactive components.
Server-client bridge	Inertia.js 3	Connects Laravel routes to Vue pages without a separate REST API.
Database	PostgreSQL	Relational storage for family, user, contribution, payment, expense, and AI records.
Styling	Tailwind CSS 4	Responsive interface styling.
Authentication	Laravel Fortify	Login, registration, password reset, email verification, and two-factor support.
Passwordless access	WebAuthn/passkeys	Strong browser-based authentication.
Payment gateway	Paystack	Online contribution payments and subscription billing.
AI integration	Laravel AI SDK	Assistant responses, tool calling, and provider integration.
Exploratory predictive method	Logistic regression	Selected method for the unevaluated prototype; no empirical predictive result.
Prototype development tools	Python 3.12, scikit-learn, NumPy, Matplotlib	Implemented validation, training and evaluation code tested with fixtures; real-data training remains future work.
Feature flags	Laravel Pennant	Controlled release of features such as AI assistance.
Testing	Pest PHP	Automated tests for models, features, policies, payments, reports, and AI tools.
Build tool	Vite	Frontend asset compilation and development server.
Version control	Git	Source tracking and collaboration.

Category	Tool/Technology	Purpose
Code quality	Laravel Pint, ESLint, Prettier	Formatting and static code quality.
Development tools	Composer, Node.js, npm	Dependency management and project tooling.

Laravel was selected because it provides mature support for routing, validation, queues, scheduled commands, notifications, policies, and Eloquent models. Vue and Inertia were selected because they allow a modern single-page experience without forcing the project to maintain a separate API layer. PostgreSQL was selected as the stated database technology because the project requires reliable relational storage, constraints, and query support. Paystack fits the Nigerian payment context. The Laravel AI SDK keeps assistant features inside the application's own permission and tool structure.

3.13 Ethical Considerations

The project handles financial and personal information, so ethical considerations are central to the design. The first concern is privacy. A family member's contribution history, payment status, phone number, email address, and role should not be exposed to another family or to unauthorised members of the same family. Tenant isolation and RBAC are therefore ethical safeguards as well as technical features.

The second concern is consent and transparency. Members should understand that their records are being stored in a digital system for family fund management. If AI summaries or assistant features are used, users should understand that the AI is a support tool and not an independent auditor or financial adviser.

The third concern is data minimisation. The system should collect only data needed for contribution management, payment processing, reminders, reporting, authentication, and authorised AI assistance. AI prompts should avoid sending unnecessary personal data to external providers.

The fourth concern is security. Password hashing, email verification, two-factor authentication, passkeys, signed invitation links, webhook validation, HTTPS deployment, and secure environment variables are required to protect users. The design should also comply with applicable Nigerian data protection expectations, including responsible processing and protection of personal data under the Nigeria Data Protection Act 2023.

The predictive component introduces a separate fairness and dignity concern. A late-payment estimate is an advisory signal for authorised officers, not a factual label about a member's character and not a basis for automatic sanctions. The interface must explain data sufficiency and model version, restrict access by role, avoid exposing another family's history, and provide an unavailable state rather than manufacturing confidence from sparse data.

The fifth concern is intellectual property. The project uses open-source frameworks and libraries such as Laravel, Vue, Inertia, Tailwind CSS, and Pest according to their licences. External services such as Paystack and AI providers are used through their documented APIs and configuration requirements.

3.14 Summary of the Chapter

This chapter described the methodology used for FamilyFunds. It adopted an applied system-development design and an Agile iterative approach because the project solves a practical problem whose requirements become clearer through design and implementation. The chapter analysed the existing manual system, introduced the proposed system, specified functional and non-functional requirements, explained the data collection method, clarified the non-survey sampling approach, and described the architecture, diagrams, database design, algorithms, activation-gated logistic regression design, tools, and ethical considerations.

The methodology shows how the project moves from the literature gap in Chapter Two to a concrete system design. Chapter Four will present how the designed modules were implemented, tested, and validated against the requirements stated in this chapter.

CHAPTER FOUR

SYSTEM IMPLEMENTATION AND TESTING

4.1 Implementation of the System Design

4.1.1 Introduction to the Chapter

This archiving revision reflects the defence feedback on the project's title and predictive scope. The main implementation concerns family-fund administration with controlled AI assistance. Predictive analytics is reported separately as an unevaluated prototype. The test results below retain their recorded August 2026 dates; revising the write-up does not constitute a new software test run.

This chapter reports how the FamilyFunds design described in Chapter Three is represented in the application and how that representation will be validated. It separates three kinds of evidence that must not be confused. The first is static implementation evidence, such as a named service, policy, migration, page, or automated test in the repository. The second is executed evidence, such as the output of a fresh test, build, browser, performance, or security run against the final revision. The third is external evidence, such as a live AI-provider response, hosted deployment check, populated screenshot, or predictive-model evaluation that depends on data or services outside the source tree.

The repository provides substantial static evidence for the conventional system modules and the predictive implementation pipeline. Fresh post-change evidence was executed on 15 August 2026. The canonical local quality gate passed 1,421 Pest tests with 6,904 assertions and 100.0% PHP coverage in 22.209 seconds; the same gate passed PHPStan, Pint, Prettier, ESLint, and the client and server-side-rendering builds. Its locked Python stage passed 66 machine-learning tests with 100% statement coverage. Focused checks additionally passed 138 Laravel payment-risk tests with 440 assertions and 100% coverage across the 16 payment-risk PHP files; the same 138 tests and 440 assertions passed against a disposable PostgreSQL database; five focused Chromium browser tests passed with 82 assertions; and a combined local payment-risk, subscription, and package-script check passed 171 tests with 517 assertions. These focused results overlap the canonical suite and are therefore not added into one artificial total. Twenty redacted running-system images were also captured from the

synthetic development tenant and are presented as Figures 4.1a–4.14. Hosted checks, live-provider verification, performance measurements, recovery evidence, and authorised-data predictive training remain Partially Met or Not Executed. Commit a699a821 is a pre-change hosted baseline and is not accepted as final Chapter Four evidence.

4.1.2 Implementation and Evidence Environment

The development environment was inspected directly for this report track. FamilyFunds remains a modular Laravel application with a Vue/Inertia interface, PostgreSQL data design, background jobs, scheduled commands, external payment and messaging integrations, and a controlled AI layer. The local computer, installed command-line versions, and disposable PostgreSQL integration environment can be reported now. The final hosted revision, deployment-service state, browser version, live provider behaviour, and authorised predictive-training environment still require external evidence.

Table 4.1: Implementation and Evidence Environment

Area	Current evidence	Purpose	Evidence status
Development computer	Apple MacBook Pro, Apple M5 Pro, 15 CPU cores, 24 GB RAM, arm64	Local development and document preparation	Observed
Operating system	macOS 26.6	Local host environment	Observed
PHP runtime	PHP 8.5.8	Laravel application runtime	Observed
Composer	Composer 2.10.1	PHP dependency management	Observed
Backend framework	Laravel 13.24.0	Routing, policies, models, jobs, services, and commands	Observed from installed dependencies
Frontend runtime and tools	Node.js 24.15.0, npm 11.14.1, Vite 8.2.1	Vue build and browser assets	Observed
Frontend frameworks	Vue 3.5.41, Inertia Vue 3.6.1, Tailwind CSS 4.3.3	Responsive application interface	Observed from installed dependencies

Area	Current evidence	Purpose	Evidence status
Testing	Pest 5.0.4 with Laravel and browser plugins; canonical local run passed 1,421 tests/6,904 assertions at 100.0% PHP coverage; focused runs passed 138 payment-risk tests/440 assertions, 5 browser tests/82 assertions, and 171 combined tests/517 assertions	Unit, feature, integration, and browser specifications	Post-change local gate passed; hosted gate pending
Database	PostgreSQL 18.4 on arm64, including a disposable migrated integration database	Persistent family and financial records	Migration smoke and focused payment-risk integration passed: 138 tests/440 assertions
Hosted environment	HTTPS application with queues, scheduler, backups, and monitoring required	Deployment and operations	Not Executed for final revision
AI provider	Provider- and model-configurable Laravel AI integration	Controlled assistant and summary generation	Live evidence Not Executed
Predictive runtime	Python 3.12.3 package plus Laravel artefact installation and scoring services	Logistic regression with training-prevalence and previous-period-late baselines	66 Python tests passed at 100% coverage; authorised-data training Not Executed

No authorised private predictive-training CSV matching the twelve-column contract is present in docs/project-report or the supplied project evidence. Consequently, no dataset row count, class distribution, coefficient, threshold, accuracy value, confusion matrix, or prediction screenshot is reported. The model must remain inactive until an authorised dataset is supplied and the PRED evidence gates in Section 4.3 are completed.

4.1.3 Design-to-Implementation Traceability

The design in Chapter Three is realised through request middleware, policies, Eloquent models, domain services, controllers, queued jobs, scheduled commands, Vue pages, and automated tests. Shared-schema multi-tenancy is not enforced by a single mechanism. The active family is resolved from the authenticated membership and route context; family-owned queries include family identifiers; policies test membership and role; validation constrains

identifiers to the current family; and feature tests exercise cross-family rejection. This layered approach is important because financial isolation would be fragile if it depended only on a hidden form value or one global query scope.

Financial correctness is similarly distributed. Monthly contribution obligations remain separate from payment receipts and allocation rows. `PaymentAllocationService` creates an immutable receipt batch, locks the family and membership records, orders incomplete contributions from oldest to newest, allocates the available amount, and limits advance allocation. Paystack processing verifies signatures and references before settlement, while reporting uses shared query logic so displayed, CSV, PDF, receipt, and scheduled outputs can be compared against the same totals.

The implementation also distinguishes authoritative application decisions from external-service observations. Paystack may attest to a transaction, a mail provider may accept a message, and an AI provider may return generated text, but none of those services owns the family ledger or its permissions. Laravel validates the external response, maps it to a known family-owned record, and applies a domain service or rejects the request. This boundary makes external failures recoverable: a provider timeout should leave a pending job or failed delivery record, not a half-written receipt. The same principle applies to the predictive component. A calculated probability is advisory evidence; it cannot create an expense, alter an obligation, block a member, or override an officer's governed decision.

The database design has grown beyond the entities shown in the original ER diagram. Membership history, payment batches, reversals, report artifacts, report deliveries, bank transactions, reconciliation periods and links, provider settlements, and retention state support requirements that became clearer during implementation. Chapter Three has been amended only where necessary to acknowledge these outcomes. Chapter Four retains the distinction between the original design and later extensions so an examiner can follow the real development path rather than a retrospective idealisation.

Table 4.2: Design-to-Implementation Traceability

Design area	Principal implementation evidence	Verification evidence located	Current conclusion
Tenant isolation	Family context middleware, membership routing, family-aware policies and queries	FamilyMembershipRoutingTest, policy tests, AI tool isolation tests	Met in the canonical local suite

Design area	Principal implementation evidence	Verification evidence located	Current conclusion
Roles and permissions	Role helpers, policies, middleware, request authorisation	RoleAccessTest, member and policy tests	Met in the canonical local suite
Contributions	Contribution model, generation command/controller, category history	Generation, status, due-date, and Phase One tests	Met in the canonical local suite
Payment allocation	PaymentAllocationService, immutable batches and allocations	Allocation, balance-first, partial and advance-payment tests	Met in the canonical local suite
Paystack	Service, webhook job/processor, settlement service	Service, webhook, ledger, fee, and idempotency tests	Partially Met; live gateway not executed
Expenses and adjustments	Controllers, policies, reversal actions, observers	CRUD, policy, model, reversal, and AI tool tests	Met in the canonical local suite
Reporting	Shared review query, private artifacts, receipts, schedules	Monthly, annual, Phase Three reporting and scheduled-report tests	Partially Met; rendered final artifacts pending
Notifications	Email, Web Push, WhatsApp channels and reminder commands	Manual reminder, channel, webhook, and notification tests	Partially Met; delivery not executed
Authentication	Fortify flows, 2FA, passkeys, password and verification settings	Authentication, passkey, 2FA, reset, and verification tests	Met in the canonical local suite; report capture pending
Controlled AI	Family assistant, sub-agents, role-aware tools, confirm-first writes	AI agent, tool, middleware, and chat feature tests	Partially Met; live provider evidence not executed
Predictive analytics	Offline Python validation/training/evaluation package; portable artefact; Laravel validation, storage and scoring services	66 Python tests at 100% coverage; 138 Laravel tests/440 assertions at 100% payment-risk-file coverage; PostgreSQL and browser reruns	Partially Met; software checks passed, but private-data training and activation Not Executed
Reconciliation extension	Imports, bank transactions, periods, matching, links, settlements	Phase Four and reconciliation unit/browser tests	Post-design extension; local automated checks passed
Retention and recovery	Archive, restore, purge, legal hold, backup configuration	Phase Three retention and backup tests	Partially Met; hosted recovery drill not executed

Reconciliation deserves explicit treatment because it was added after the original Chapter Three requirement set. Iterative development exposed a gap between application records and external bank statements: a verified receipt, expense, adjustment, or provider settlement could be internally valid yet still require comparison with an imported transaction. The later reconciliation workspace introduced import batches, bank transactions, controlled matching, immutable links, closeable periods, and provider-settlement grouping. This is an Agile post-design extension that improves auditability; it is not presented as an original FR that existed before implementation.

4.2 Module Integration and Coding

4.2.1 Tenancy, Membership, and Role Governance

FamilyFunds supports users who may belong to more than one family while operating within one current family context at a time. Membership records carry the family-specific role, contribution category, lifecycle state, and display information. Route middleware resolves the selected family, and policies make the final decision for protected records. An ordinary member can inspect personal contributions and payment history, while family administrators and financial secretaries receive wider operational views according to their duties. Platform administration is kept separate from family operations.

The membership model avoids storing all authority as one global property of the user. A person can therefore be a financial secretary in one family and an ordinary member in another without one role leaking across the boundary. Category assignments are dated so that a future contribution-rate change does not rewrite the amount that applied to an older obligation. Invitation acceptance attaches the user to the intended family only after token, expiry, identity, role, and category rules have been checked. Archive state is also membership-aware, allowing a person's relationship with one family to change without deleting the global account or unrelated memberships.

This design introduces more joins than a single `family_id` column on the user record, but it better represents the domain and supports auditability. Compatibility mirrors remain for earlier records, while backfill and routing tests protect the transition. The final evidence run must exercise both a single-family member and a person with multiple memberships because cross-context behaviour is where role leakage would be most likely.

Listing 4.1 shows the small but important context-binding step. It does not, by itself, authorise access; downstream membership checks, policies, validation, and scoped queries remain necessary.

Listing 4.1: Family Context Binding

```
if ($user) {  
    $family = $user->currentFamily ?? $user->family;  
  
    if ($family instanceof Family) {  
        app()->instance('current-family', $family);  
        app()->instance(Family::class, $family);  
    }  
}
```

4.2.2 Contributions and Oldest-Balance-First Allocation

Contribution categories define monthly expectations for memberships. Generation creates dated obligations, while status logic distinguishes unpaid, partially paid, paid, and overdue records. A receipt is modelled separately from its allocation lines, allowing one incoming amount to satisfy several months while preserving the amount, recorder, source, reference, and receipt number. The allocation service uses a transaction and row locks to prevent concurrent writes from producing inconsistent balances. Existing incomplete contributions are ordered chronologically; any permitted remainder can create future obligations only within the defined advance-payment limit.

Listing 4.2: Oldest-Balance-First Payment Allocation

```
$contributions = Contribution::query()
->forUser($member)
->where('family_id', $family->id)
->where(fn ($query) => $query->incomplete())
->oldestFirst()
->lockForUpdate()
->get();

foreach ($contributions as $contribution) {
    $remainingAmount = $this->allocateLine(
        $batch,
        $contribution,
        $remainingAmount,
        $paidAt,
        $recordedBy,
        $notes,
        $replaces,
    );

    if ($remainingAmount === 0) {
        return 0;
    }
}
```

The listing omits receipt creation and future-period handling so that the core ordering decision remains visible. The full implementation also validates family membership, uses idempotency keys, assigns sequential receipt numbers, supports corrections through replacement batches, and rejects money that would exceed the six-month advance limit.

4.2.3 Paystack, Expenses, Adjustments, Reports, and Notifications

Paystack is integrated as an external processor rather than the authority for family balances. A transaction is initialised within the application, and a signed callback or queued webhook is matched to a known reference before the settlement service is invoked. Duplicate success events must not create duplicate receipts. Amount mismatches, unknown references, missing data, and invalid signatures follow explicit rejection or ignore paths.

Listing 4.3: Paystack Webhook Reference Verification

```
$reference = $data['reference'] ?? null;

if (! is_string($reference) || $reference === '') {
    return;
}

if (! PaystackTransaction::query()
    ->where('reference', $reference)
    ->exists()) {
    Log::info('Paystack webhook ignored an unknown transaction reference.',
    [
        'reference' => $reference,
    ]);
    return;
}

$this->settlementService->settle($reference, $data);
```

Expenses and fund adjustments use immutable financial records plus explicit reversal or correction actions. Reports draw from contribution, receipt, allocation, expense, adjustment, and reversal data. Private artifacts store their family, requester, type, format, normalised filter hash, path, size, and expiry. CSV cells are sanitised to reduce spreadsheet-formula risk; PDFs and receipts use A4 output; signed recipient links expire. Scheduled jobs can create and deliver the same report definitions without introducing a separate calculation path.

Separating receipt batches from allocation lines also clarifies reports. A family's cash receipt should appear once at its full value, while a member statement may need the individual month-by-month allocation lines. Reversals must remove the effective financial impact without deleting the historical record. The shared review service is intended to keep these interpretations consistent across screen, CSV, PDF, receipt, and scheduled delivery. The Phase Three reporting specifications are valuable static evidence because they compare these surfaces, but the final report must still render and inspect representative PDF and CSV outputs after all current changes.

Report artifacts are private by default. An authenticated, authorised requester may download an artifact through policy checks, while a recipient link uses a time-limited signature. The path is stored on the local private disk rather than a public asset directory. This matters academically because “a report was generated” is incomplete evidence if the report leaks another family or can be guessed indefinitely. The final screenshot set should show the report

interface and access controls without exposing real family names, phone numbers, email addresses, bank references, or personal contribution histories.

Reminder workflows select unpaid or partially paid obligations and support the configured email, browser-push, and WhatsApp-related channels. Delivery state is operational evidence and cannot be established from source alone. The current report therefore records channel and command coverage as partial while leaving actual message delivery for the final evidence run.

4.2.4 Controlled AI Assistant and Report Summaries

The assistant coordinates specialist agents for contribution, expense, balance, member-status, invitation, generation, and write workflows. Tools query current family data rather than relying on model memory. Their availability is filtered by role. Write operations follow a preview-and-confirm pattern: the initial tool call returns a human-readable description and the same details needed for execution; only a later confirmed call can mutate records. This reduces accidental changes and keeps Laravel policies, validation, and domain services in authority over the language model.

Listing 4.4: Confirm-First AI Write Governance

```
if (! $confirmed) {
    return json_encode([
        'status' => 'confirmation_required',
        'message' => "I'll record a payment of {$formattedAmount} for
{$member->name}. Please confirm to proceed.",
        'details' => [
            'member' => $member->name,
            'amount' => $amount,
            'paid_at' => $paidAt,
        ],
    ], JSON_THROW_ON_ERROR);
}
```

Static feature tests cover role-aware tools, family isolation, validation, preview responses, and confirmed execution. They do not prove the quality of a live model response. No provider-backed conversation or final report-summary example was executed for this document revision, so live AI relevance, latency, and wording remain Not Executed.

4.2.5 Reconciliation as a Post-Design Extension

The reconciliation workspace accepts bank-transaction imports, preserves import and period state, proposes candidate internal records, supports explicit links, and locks closed periods.

Exact automatic matching is deliberately narrow. A credit is automatically linked only where reference and amount produce one unique unmatched candidate; ambiguity produces suggestions for officer review. Date-and-amount proximity is useful for suggestions but is not sufficient for an automatic authoritative link.

Provider settlements are distinguished from member receipts so that a net bank deposit is not incorrectly matched to a gross contribution. Closed-period guards prevent late mutation unless an authorised reopening process is completed. These behaviours extend the original fund-management design while maintaining its role, tenant, audit, and immutability principles.

4.2.6 Unevaluated Predictive Prototype

This section documents supplementary engineering work, not an operational prediction capability. No authorised historical dataset was available, so the study produced no trained model for practical use and no empirical accuracy result. The seeded records used in the demonstration test application behaviour; they do not establish predictive performance. The following code and protocol describe the preparation completed for a possible future study.

The predictive component is implemented as two connected parts. An offline Python 3.12 package accepts a separately authorised CSV and provenance file, validates them, creates point-in-time features, performs a chronological split, fits and evaluates the approved model, and emits a portable JSON artefact and checksummed evidence files. Laravel validates that artefact again, stores it on a private disk, records immutable model-version metadata, permits activation only when all gates pass, and scores family-owned contribution targets. This separation keeps private training material and Python dependencies out of web requests while allowing deterministic PHP inference.

The selected method for the prototype is logistic regression. The intended target is a binary overdue/on-time outcome for a contribution period: an obligation is overdue when the system still recorded an outstanding balance at the end of its due date. The fourteen implemented features are expected amount, days until due, calendar month and quarter, number of previous mature periods, three-period, six-period and lifetime on-time rates, previous partial-payment rate, mean and median recorded settlement delay, previous outstanding count and amount, and overdue streak. The outstanding-amount feature is operationally the due-date residual summed only for prior obligations still unsettled strictly before the scoring cutoff; this keeps offline and Laravel reconstruction identical under the locked twelve-column export. All

features are constructed from data recorded strictly before the target obligation's creation-time cutoff. Direct identity attributes and target-period outcome data are excluded.

The input contract is deliberately strict. The uncommitted source must be `storage/app/private/payment-risk/source/member-periods.csv` and contain exactly: `family_key`, `member_key`, `obligation_key`, `period_start`, `obligation_created_at`, `due_date`, `expected_amount_minor`, `amount_paid_by_due_minor`, `first_payment_recorded_at`, `fully_paid_recorded_at`, `payment_count_by_due`, and `is_backfilled`. The three opaque pseudonymous keys support grouping and split integrity only; they are never predictors. The CSV digest must match its provenance JSON. Provenance records whether consent, de-identification and point-in-time correctness were confirmed. Because this fixed export cannot express the time-varying effect of later reversals, it also requires the explicit `system_known_by_cutoff_no_reversed_allocations` policy: histories containing reversed allocations must be excluded upstream before the minimum-data gate is applied. Backfilled obligations, incomplete periods, PII or extra columns, contradictory payment timestamps, duplicate obligations/member-periods, negative amounts, impossible chronology, non-mature outcomes, and obligations created fewer than seven days before their due date are rejected. The default quality gates require at least 500 valid rows, 50 distinct member histories, 12 complete periods, and 100 rows in each outcome class. Synthetic provenance may exercise the software path but is always activation-ineligible and cannot support real-world performance claims. These are implemented rules, not claims about the unavailable private dataset.

The label and prediction cutoff are fixed before data is examined. A feature such as “days late for the target period” would leak the answer because it becomes known only after the outcome window. By contrast, the previous mature periods' outcomes, settlement delays and outstanding amounts are available at the obligation-creation cutoff. The implemented artefact stores the ordered feature schema, standardisation means and scales, coefficients, intercept, threshold, training window, data-quality report, evaluation summary and human-readable factor labels so that training and inference cannot silently disagree.

Chronological evaluation is required because a random row split could place later records for the same member in training while earlier records appear in testing. The implementation keeps whole contribution periods together and assigns the earliest 60% to training, the next 20% to validation, and the latest 20% to held-out evaluation. Standardisation and the class-

balanced L2 logistic regression are fitted only on training data. The decision threshold is selected only on validation data by maximising overdue-class F1 subject to at least 0.70 overdue recall, with precision and threshold as deterministic tie-breakers. The latest partition is not consulted during fitting or selection.

Listing 4.5: Logistic Regression Training Code for the Unevaluated Prototype

```
scaler = StandardScaler()
training_features = scaler.fit_transform(split.training.features)
classifier = LogisticRegression(
    C=1.0,
    class_weight="balanced",
    max_iter=2_000,
    penalty="l2",
    random_state=RANDOM_SEED,
    solver="liblinear",
)
classifier.fit(training_features, split.training.targets)
validation_probabilities = classifier.predict_proba(
    scaler.transform(split.validation.features)
)[: , 1]
threshold = select_validation_threshold(
    split.validation.targets,
    validation_probabilities,
)
held_out_probabilities = classifier.predict_proba(
    scaler.transform(split.held_out.features)
)[: , 1]
```

Two baselines are computed on the same held-out partition. The training-prevalence baseline assigns every held-out record the late-class prevalence learned from the training partition. The previous-period-late baseline predicts late when the feature snapshot contains a positive overdue streak. Evaluation records accuracy, balanced accuracy, overdue precision, recall and F1, ROC-AUC, PR-AUC, Brier score, confusion matrix, and 95% bootstrap intervals. The training package also writes confusion-matrix, ROC, precision-recall, calibration and coefficient charts. No populated chart is included in this report because the authorised dataset has not been supplied.

Listing 4.6: Planned Baseline Comparison and Activation Gates

```
training_prevalence = float(np.mean(split.training.targets))
prevalence_probabilities = np.full_like(
    split.held_out.targets,
    training_prevalence,
    dtype=np.float64,
)
prevalence_baseline = _binary_metrics(
    split.held_out.targets,
    prevalence_probabilities,
    0.5,
)
overdue_streak_index = FEATURE_NAMES.index("overdue_streak")
previous_period_probabilities = (
    split.held_out.features[:, overdue_streak_index] > 0
).astype(np.float64)
previous_period_baseline = _binary_metrics(
    split.held_out.targets,
    previous_period_probabilities,
    0.5,
)

activation_checks = {
    "consented_anonymized_provenance": provenance.source_type ==
"consented_anonymized"
    and provenance.consent_confirmed,
    "data_quality_gate_passed": data_quality["gate_passed"],
    "validation_recall_at_least_0_70": validation["recall_overdue"]
    >= MINIMUM_VALIDATION_RECALL,
    "held_out_f1_beats_both_baselines": held_out["overdue_f1"] >
baseline_f1,
    "held_out_balanced_accuracy_above_0_5": held_out["balanced_accuracy"] >
0.5,
    "held_out_brier_beats_training_prevalence": held_out["brier_score"]
    < prevalence_baseline["brier_score"],
}
```

Activation is intentionally stricter than successful fitting. The training command can produce a candidate artefact, but the Laravel installer continues to reject activation unless the Python decision and independent PHP checks agree. The private artefact is identified by schema and model versions, ordered features, SHA-256 digest, training window, trained timestamp, threshold, dataset summary and metrics. The repository refuses an overwritten version with a different digest, prevents mutation or deletion of installed metadata and predictions, accepts only one active version, revalidates the artefact before scoring, and rejects a stale training window.

Runtime scoring also handles sparse history explicitly. Fewer than three mature periods produces an unavailable result; three to five periods use the training-prevalence pooled baseline instead of an individual probability; six to eleven periods are labelled experimental;

and twelve or more periods receive a standard logistic advisory. The application records the immutable cutoff, history tier, feature-snapshot hash, model version, advisory band and top coefficient contributions. These results are designed for routine or priority review only and cannot directly penalise a member or mutate the ledger.

The implemented interface is officer-only through the report permission, subscription entitlement and predictive feature flag. Its readiness state reports why no active model can be used, while a ready state exposes the version, training window, threshold, period/band filters, summary counts, member history tier, advisory band and explanatory factors. The page states that the output is not a credit score and cannot send reminders, restrict access, change contribution terms or automate an action. A focused Chromium run passed five tests with 82 assertions, covering administrator and financial-secretary access, member denial, tenant switching, refresh-safe filters, synthetic populated and unavailable states, keyboard use, dark mode and a 390-pixel layout without JavaScript, accessibility or smoke errors. The running-system unavailable state is shown in Figure 4.12. This verifies the interface contract and fail-closed presentation, but it does not substitute for an authorised active-model result.

No authorised private predictive-training CSV, executed training output, fitted coefficient set, candidate artefact, active model, or real-model populated predictive screenshot was available to this report task. The populated state used by the browser test was synthetic test state and is not model-performance evidence. It would be misleading to include representative coefficients or fabricated results. Listings 4.5 and 4.6 are repository implementation excerpts, not evidence that training or activation occurred. PRED-01–PRED-10 define the exact evidence that must be added after an authorised dataset is available.

4.2.7 Interface and Screenshot Evidence Register

The captured interface sequence follows the user journey and the financial control flow rather than presenting disconnected pages. All twenty assets came from the operating local application and its synthetic development tenant. They contain no production records or credentials, and no payment, reminder, schedule, reconciliation link or external-provider request was submitted during capture. Where a view depends on evidence that was unavailable, the figure shows the genuine fail-closed or read-only state instead of a mock result.

Table 4.3: Chapter Four Interface Evidence Register

Figure	Required running-system view	Primary requirements	Current status
4.1	Login, password confirmation, and account-security settings	FR1, NFR3	Executed; three panels
4.2	Family administrator dashboard	FR2, FR15	Executed
4.3	Member, role, and category management	FR3–FR5	Executed
4.4	Contribution register with paid, partial, unpaid, and overdue states	FR6–FR7	Executed
4.5	Oldest-first allocation, member statement, and receipt	FR8, FR11	Executed; three panels
4.6	Paystack self-pay review state	FR9–FR10	Interface executed; live provider transaction Not Executed
4.7	Expense and fund-adjustment records	FR12–FR13	Executed; two panels
4.8	Reports, exports, schedules, and generated financial output	FR14	Executed; two panels
4.9	Seeded contribution-reminder notification	FR16	Interface executed; no delivery request sent
4.10	Subscription entitlements and family limits	FR17	Executed
4.11	Existing controlled-AI conversation	FR18–FR19	Interface executed; no live provider call made
4.12	Predictive model-unavailable state	FR20, NFR13–NFR14	Executed; populated model result Not Executed
4.13	Reconciliation workspace and settlement candidate	Post-design extension	Executed; no statement import or link submitted
4.14	Responsive member dashboard at 390 pixels	NFR6	Executed

Figures 4.1a–4.1c record the authentication boundary from the initial sign-in screen through password confirmation to the security controls. The sequence matters because sensitive settings are not presented as an extension of a public login form; the authenticated user must re-establish intent before changing password, two-factor or passkey settings.

Figure 4.1a: Login Entry for the Synthetic Demonstration Tenant



Log in to your account
Enter your email and password below to log in

[Sign in with a passkey](#)

OR CONTINUE WITH EMAIL

Email address

Password [Forgot password?](#)

☐ Remember me

Log in

Don't have an account? [Sign up](#)

Figure 4.1b: Password Confirmation Before a Sensitive Setting



Confirm your password
This is a secure area of the application. Please confirm your password before continuing.

Password

Confirm Password

Figure 4.1c: Account Security, Two-Factor and Passkey Settings

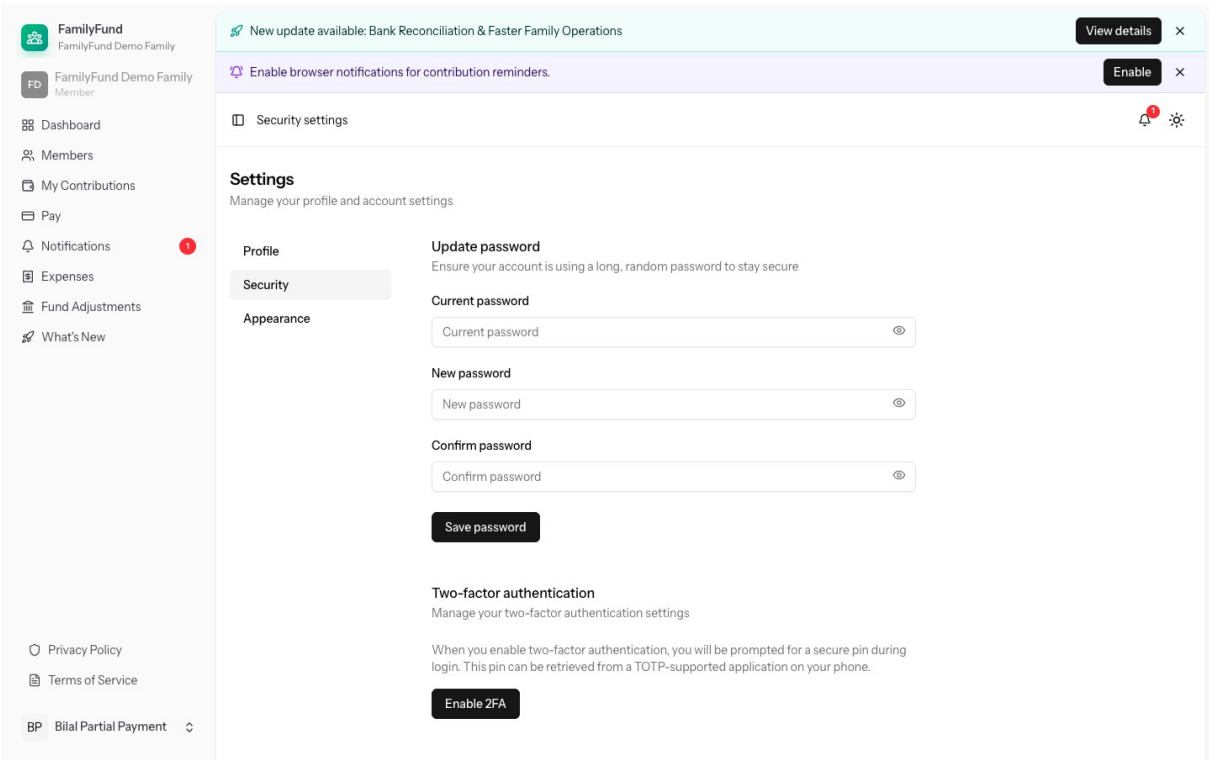


Figure 4.2 shows the administrator's current-family dashboard, where contribution and balance information is summarised without exposing another tenant. Figure 4.3 then shows the management context in which roles and contribution categories are assigned. Together they connect the tenancy and RBAC design to the officer's normal operating view.

Figure 4.2: Family Administrator Dashboard

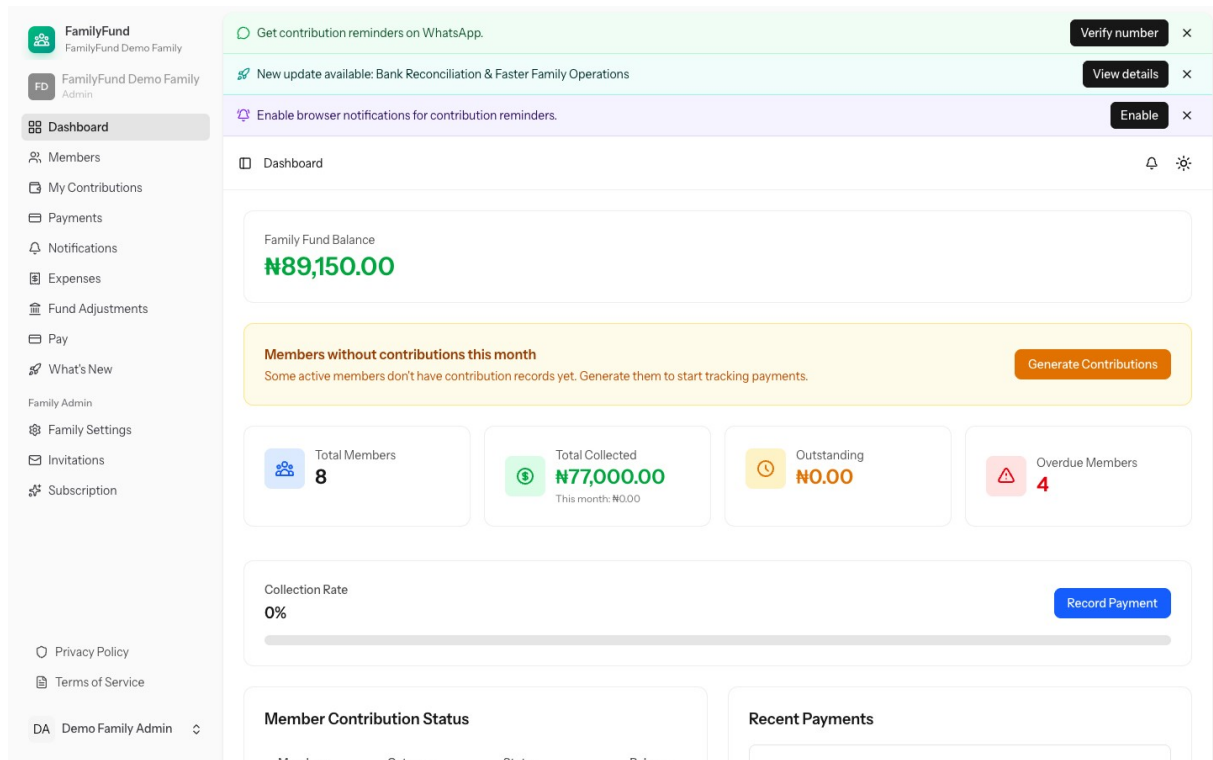
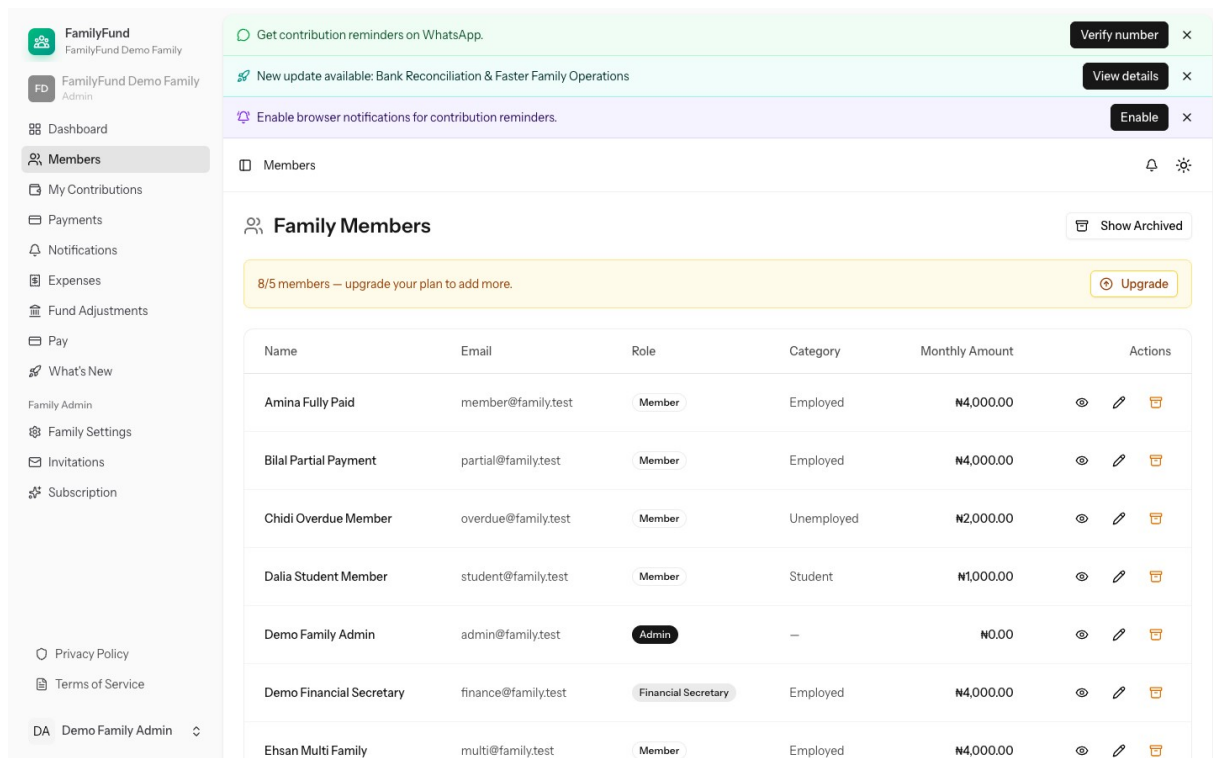


Figure 4.3: Member, Role and Contribution-Category Management



Figures 4.4 and 4.5 show the ledger rule from both sides. The register exposes paid, partial, unpaid and overdue obligations, while the payment screen explains how an amount is applied to the oldest open periods. This visible ordering supports the same deterministic rule exercised by the automated tests.

Figure 4.4: Contribution Register and Payment States

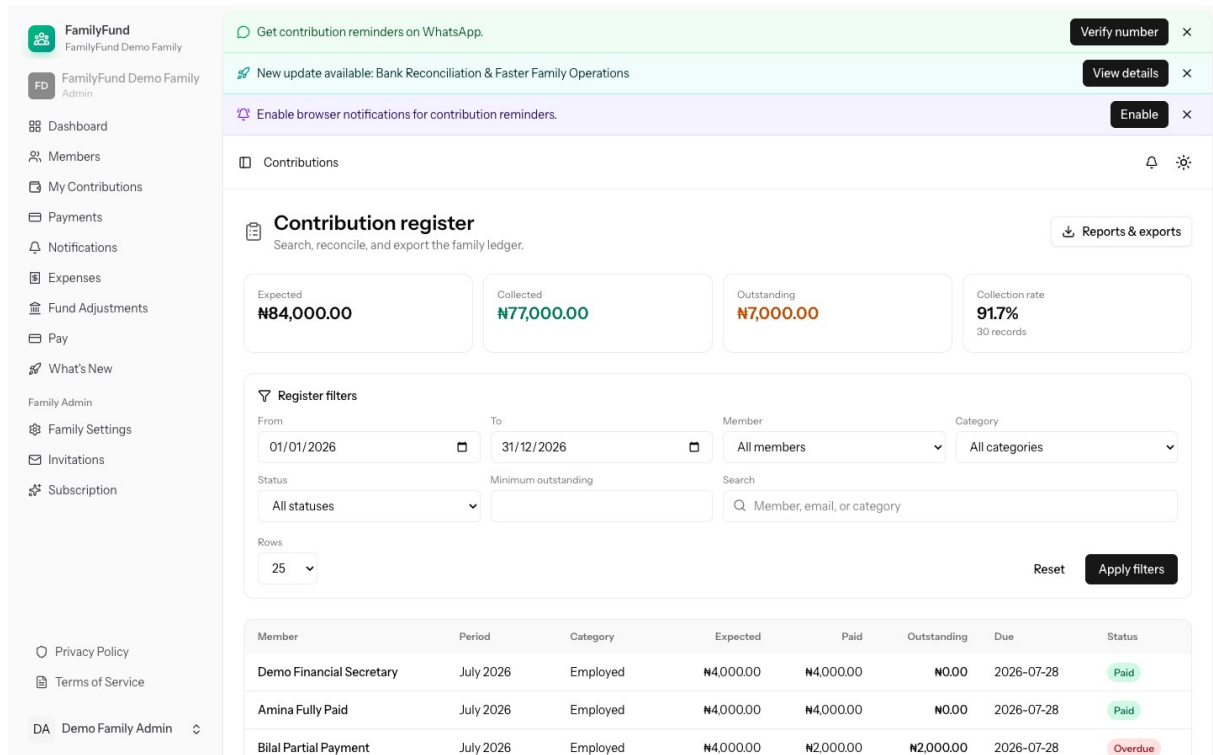


Figure 4.5: Oldest-First Payment Allocation Review

The screenshot displays the 'Record Payment' page for a family member named Bilal. The interface includes a sidebar with navigation options like Dashboard, Members, My Contributions, Payments, Notifications, Expenses, Fund Adjustments, Pay, What's New, Family Admin, Family Settings, Invitations, and Subscription. The main content area shows the 'Record Payment for Bilal Partial Payment' with a monthly contribution of ₦4,000.00. A yellow box highlights 'Pending Contributions' for July 2026, indicating an overdue amount of ₦2,000.00. Below this, there are input fields for the amount (with a monthly contribution reference), a dropdown for the target month (set to 'Auto-allocate to oldest unpaid month'), a payment date field (15/08/2026), and a payment method dropdown (set to 'Cash'). A reference field is also present for optional bank or receipt reference.

FamilyFund
FamilyFund Demo Family

FD FamilyFund Demo Family Admin

Dashboard

Members

My Contributions

Payments

Notifications

Expenses

Fund Adjustments

Pay

What's New

Family Admin

Family Settings

Invitations

Subscription

Privacy Policy

Terms of Service

DA Demo Family Admin

Get contribution reminders on WhatsApp. Verify number x

New update available: Bank Reconciliation & Faster Family Operations View details x

Enable browser notifications for contribution reminders. Enable x

Dashboard > Record Payment

Record Payment for Bilal Partial Payment
Employed - ₦4,000.00/month

Pending Contributions
July 2026 Overdue ₦2,000.00 remaining
Payments will automatically be applied to the oldest incomplete month first.

Amount (₦)
Enter amount in ₦

Monthly contribution: ₦4,000.00

1 Month (₦4,000.00) 2 Months (₦8,000.00) 3 Months (₦12,000.00)
6 Months (₦24,000.00)

Target Month (Optional)
Auto-allocate to oldest unpaid month

Leave empty to auto-allocate to the oldest unpaid month.

Payment Date
15/08/2026

Payment Method
Cash

Reference (Optional)
Bank or receipt reference

Figures 4.6a–4.6c follow the resulting member evidence. The statement presents the obligation history, the receipt records how one batch was split across periods, and the Paystack page shows the self-pay review state before any external request. The last panel is interface evidence only; no live transaction was initiated during capture.

Figure 4.6a: Member Contribution Statement

Member Statement

FamilyFund Demo Family · Generated 15 Aug 2026, 08:54 UTC

Period: 2026-01-01 to 2026-12-31						
Expected: 18,000.00 · Collected: 16,000.00 · Outstanding: 2,000.00						
Member	Period	Category	Expected	Paid	Outstanding	Status
Bilal Partial Payment	July 2026	Employed	4000	2000	2000	Overdue
Bilal Partial Payment	June 2026	Employed	4000	4000	0	Paid
Bilal Partial Payment	May 2026	Employed	4000	4000	0	Paid
Bilal Partial Payment	April 2026	Employed	4000	4000	0	Paid
Bilal Partial Payment	March 2026	Student	1000	1000	0	Paid
Bilal Partial Payment	February 2026	Student	1000	1000	0	Paid

Private financial record · #

Figure 4.6b: Receipt Showing Split Allocation Across Periods

Payment Receipt

FamilyFund Demo Family
Receipt #000007

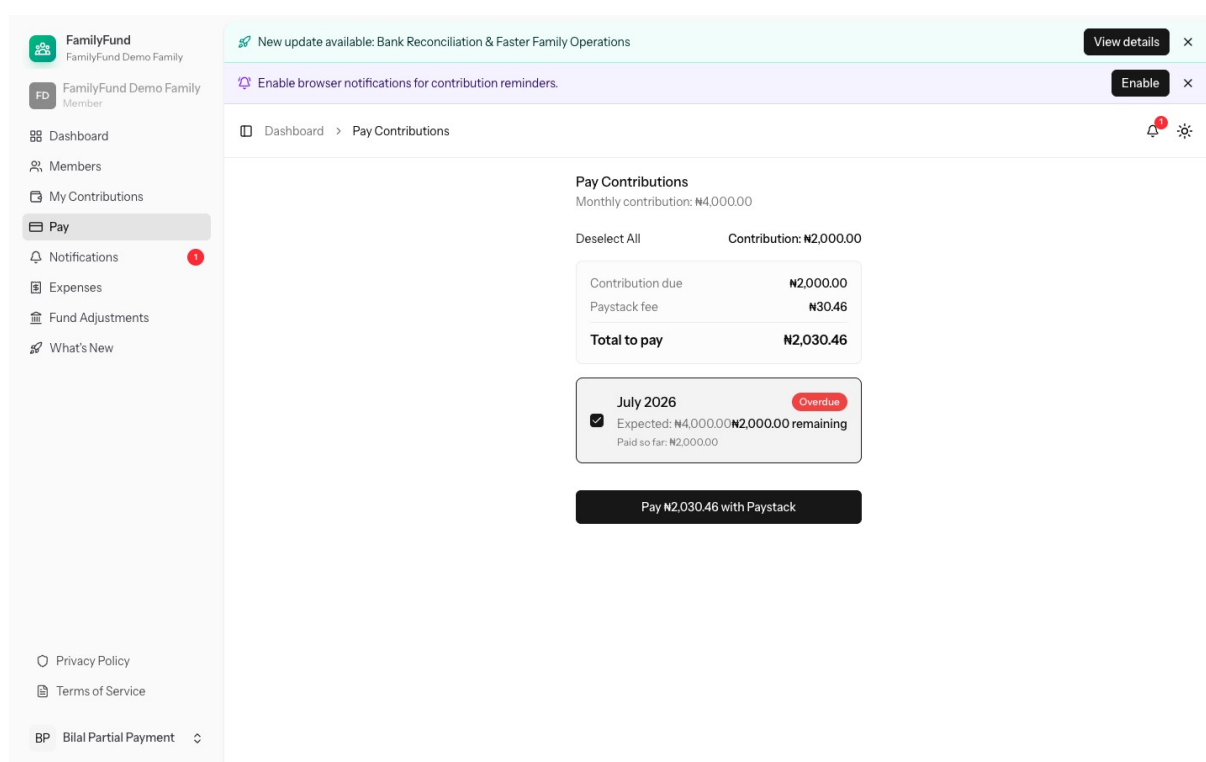
RECEIVED FROM	PAYMENT DATE
Amina Fully Paid	10 Feb 2026
METHOD	REFERENCE
Bank Transfer	DEMO-SPLIT-202602

₦ 8,000.00

Contribution Period	Amount
February 2026	₦ 4,000.00
March 2026	₦ 4,000.00

Generated 15 Aug 2026, 08:46 UTC · This receipt reflects the immutable posted payment batch.

Figure 4.6c: Paystack Self-Pay Review State Before Submission



Figures 4.7a and 4.7b show that spending and balance corrections are recorded as separate operations rather than being hidden inside a contribution entry. This separation supports clearer authorisation, reversal and audit treatment.

Figure 4.7a: Expense Register

FamilyFund
FamilyFund Demo Family

FamilyFund Demo Family
Admin

Dashboard

Members

My Contributions

Payments

Notifications

Expenses

Fund Adjustments

Pay

What's New

Family Admin

Family Settings

Invitations

Subscription

Privacy Policy

Terms of Service

DA Demo Family Admin

Get contribution reminders on WhatsApp.
Verify number
x

New update available: Bank Reconciliation & Faster Family Operations
View details
x

Enable browser notifications for contribution reminders.
Enable
x

Expenses

Expenses

+ Record Expense

Date	Description	Amount	Recorded By
6 Jul 2026	Demo: Banking and transfer fees	-N750.00	Demo Financial Secretary
4 Jul 2026	Demo: Corrected stationery expense	-N900.00	Demo Financial Secretary
4 Jul 2026	Demo: Incorrect stationery expense Reversed	-N9,000.00	Demo Financial Secretary
24 Jun 2026	Demo: Cancelled event deposit Reversed	-N5,000.00	Demo Financial Secretary
20 Jun 2026	Demo: Member welfare support	-N2,200.00	Demo Financial Secretary
18 May 2026	Demo: Community meeting venue	-N3,500.00	Demo Financial Secretary

Figure 4.7b: Fund-Adjustment Register

FamilyFund
FamilyFund Demo Family

FamilyFund Demo Family
Admin

Dashboard

Members

My Contributions

Payments

Notifications

Expenses

Fund Adjustments

Pay

What's New

Family Admin

Family Settings

Invitations

Subscription

Privacy Policy

Terms of Service

DA Demo Family Admin

Get contribution reminders on WhatsApp.
Verify number
x

New update available: Bank Reconciliation & Faster Family Operations
View details
x

Enable browser notifications for contribution reminders.
Enable
x

Fund Adjustments

Fund Adjustments

+ Record Adjustment

Fund adjustments record signed corrections to the family fund. Use a positive amount for money in and a negative amount for money out.

Date	Description	Amount	Recorded By
14 Jun 2026	Demo: Bank charge adjustment	-N500.00	Demo Financial Secretary
14 Feb 2026	Demo: Corrected opening adjustment	+N20,000.00	Demo Financial Secretary
14 Feb 2026	Demo: Incorrect opening adjustment Reversed	+N25,000.00	Demo Financial Secretary
14 Jan 2026	Demo: Opening balance Reversed	+N100,000.00	Demo Financial Secretary

Figures 4.8a and 4.8b connect report controls to a rendered financial output. The first panel shows filters, export choices and schedule management; the second shows the generated report based on the same underlying family records. This is stronger evidence than an isolated download button because the displayed result can be compared with the application totals.

Figure 4.8a: Reports, Exports and Schedule Controls

The screenshot displays the 'Reports & exports' section of the FamilyFund application. The interface includes a sidebar with navigation options like Dashboard, Members, My Contributions, Payments, Notifications, Expenses, Fund Adjustments, Pay, AI Assistant, Reconciliation, Reports (highlighted), What's New, Family Admin, Family Settings, Invitations, Subscription, Privacy Policy, and Terms of Service. The main content area features a 'Generate report' section with filters for Report type (Contribution Register), Format (PDF), From date (01/01/2026), To date (31/12/2026), Member (All members), Status (All statuses), and a 'Generate & download' button. Below this is a 'Schedule delivery' section with fields for Schedule name, Report type (Contribution Register), Format (PDF), Frequency (Monthly), Run first at (15/08/2026, 01:00 PM), Recipients (email@example.com, 974...), and Channels (Email checked, WhatsApp unchecked), with a 'Create schedule' button. The top of the page shows notification banners for WhatsApp reminders, a new update, and browser notifications.

Figure 4.8b: Generated Financial Report Output

Fund Statement

FamilyFund Demo Family · Generated 14 Jul 2026, 14:40 UTC

Period: 2026-01-01 to 2026-12-31

Opening Balance: 0.00 · **Posted Payments:** 77,000.00 · **Adjustments:** 19,500.00 · **Expenses:** 7,350.00 · **Reversals:** 6.00 · **Closing Balance:** 89,150.00
· **Reconciliation Variance:** 0.00

Statement Item	Amount	Count
Opening balance	0	—
Posted payments	77000	—
Adjustments	19500	—
Expenses	-7350	—
Reversals posted	0	6
Closing balance	89150	—
Reconciliation variance	0	—

Private financial record · [N](#)

Figure 4.9 contains an existing seeded reminder notification and does not claim a newly delivered message. Figure 4.10 shows the plan-entitlement comparison with Growth

identified as the current assigned plan, demonstrating how paid features are communicated after the shared-plan defect discovered during capture was corrected and regressed.

Figure 4.9: Seeded Contribution-Reminder Notification

The screenshot displays the FamilyFund user interface. On the left is a sidebar with navigation links: Dashboard, Members, My Contributions, Pay, Notifications (highlighted with a red badge), Expenses, Fund Adjustments, and What's New. At the bottom of the sidebar are links for Privacy Policy, Terms of Service, and a user profile for 'BP Bilal Partial Payment'. The main content area features a top banner with a 'New update available: Bank Reconciliation & Faster Family Operations' and a notification to 'Enable browser notifications for contribution reminders'. Below this is the 'Notification Center' header, followed by a summary row with four boxes: 'Total' (1), 'Unread' (1), 'Reminders' (1), and 'Follow-ups' (0). To the left of the timeline are filters for 'STATUS' (All, Unread, Read) and 'TYPE' (All types, Reminders, Follow-ups). The central 'Timeline' section shows one item: a 'Contribution reminder' for 'July 2026 contribution of ₦2,000 is coming due.' with a 'Mark read' button. The notification includes details for 'FamilyFund Demo Family' and a due date of 'Due Jul 28, 2026'.

Figure 4.10: Current Growth Plan and Feature Entitlements

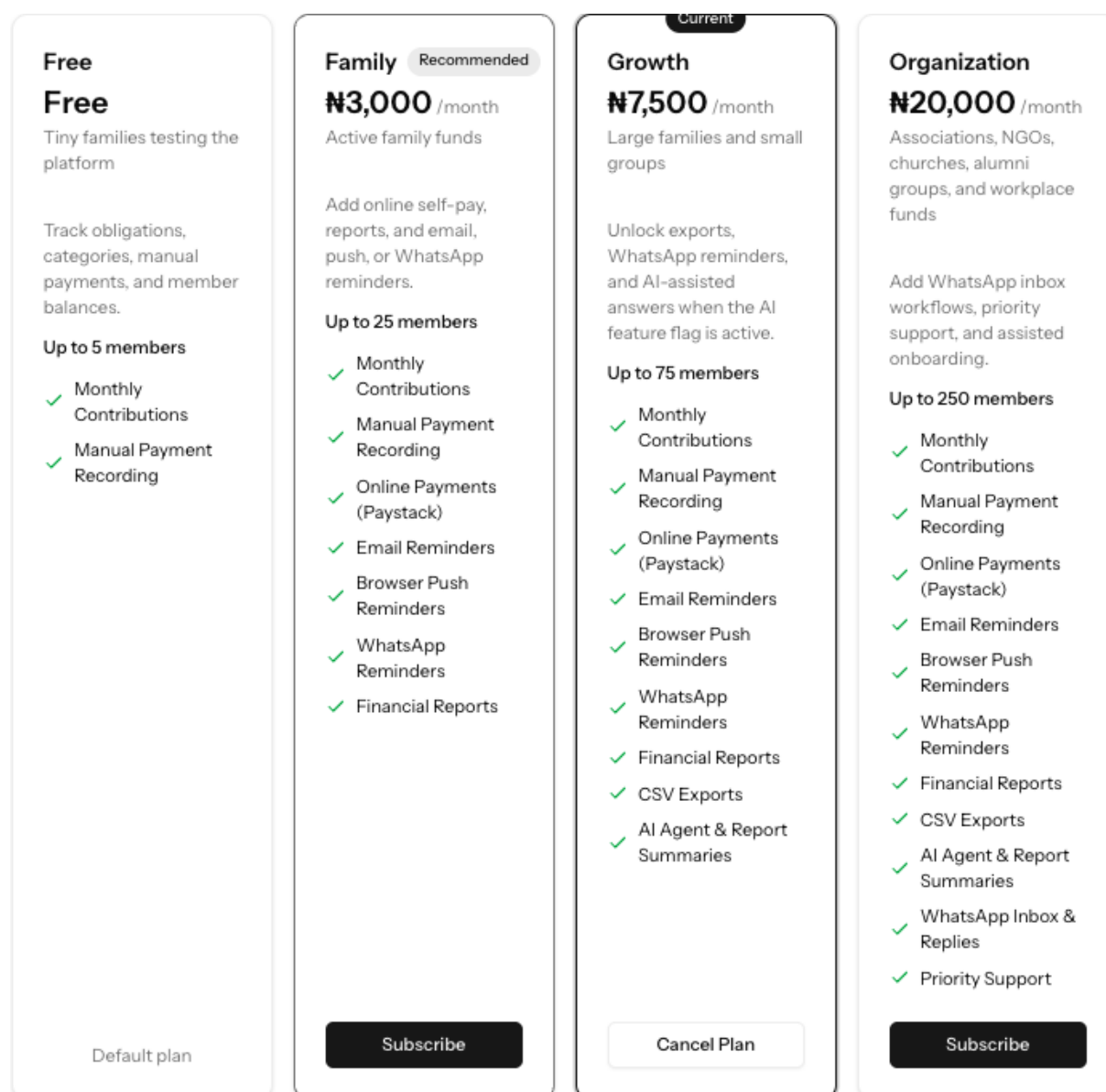


Figure 4.11 shows an existing seeded assistant conversation in the controlled interface. It demonstrates the conversation layout and family context, but it is not counted as a live-provider acceptance result because no provider request was made during this evidence pass.

Figure 4.11: Existing Controlled-AI Conversation in the Synthetic Tenant

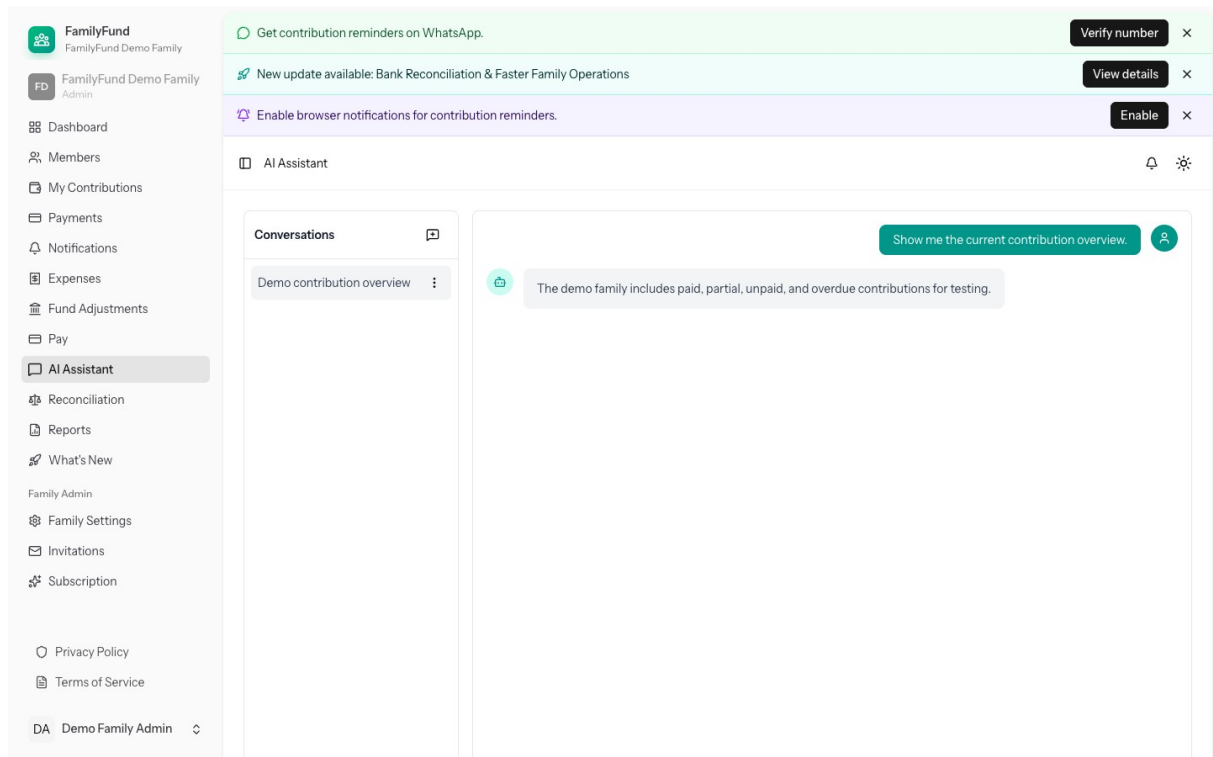
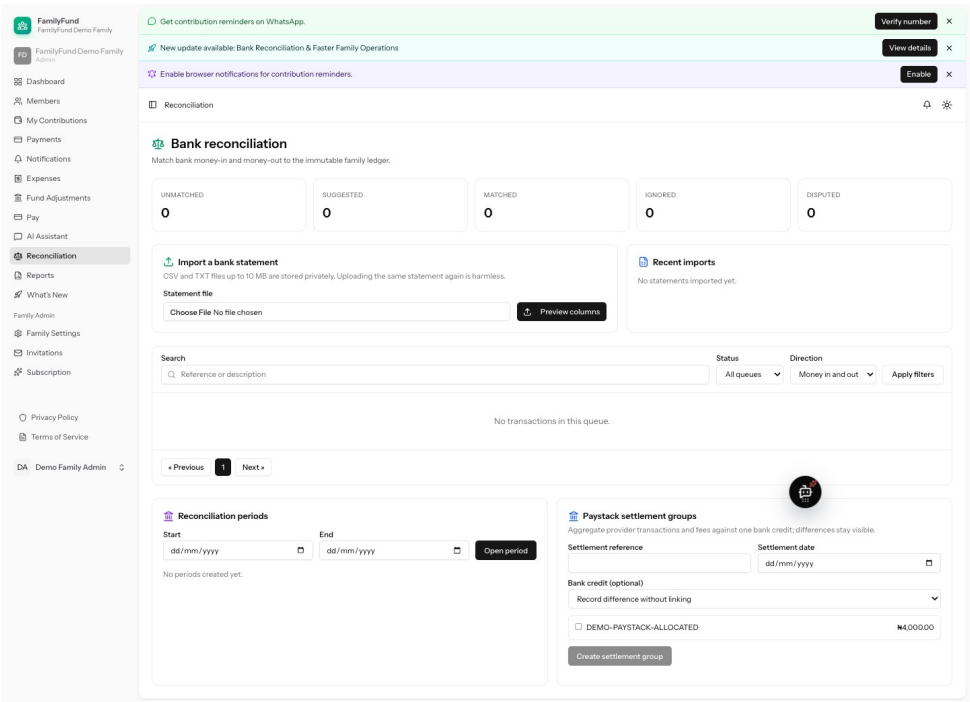


Figure 4.12 records the predictor's actual fail-closed response: without an installed, validated and active private-data model, the page reports that payment-risk insights are unavailable. Figure 4.13 shows the reconciliation workspace and an existing settlement candidate without importing a statement or creating a link. These are useful boundary results because they show what the system refuses to infer or mutate when prerequisite evidence is missing.

Figure 4.12: Predictive Analytics Model-Unavailable State

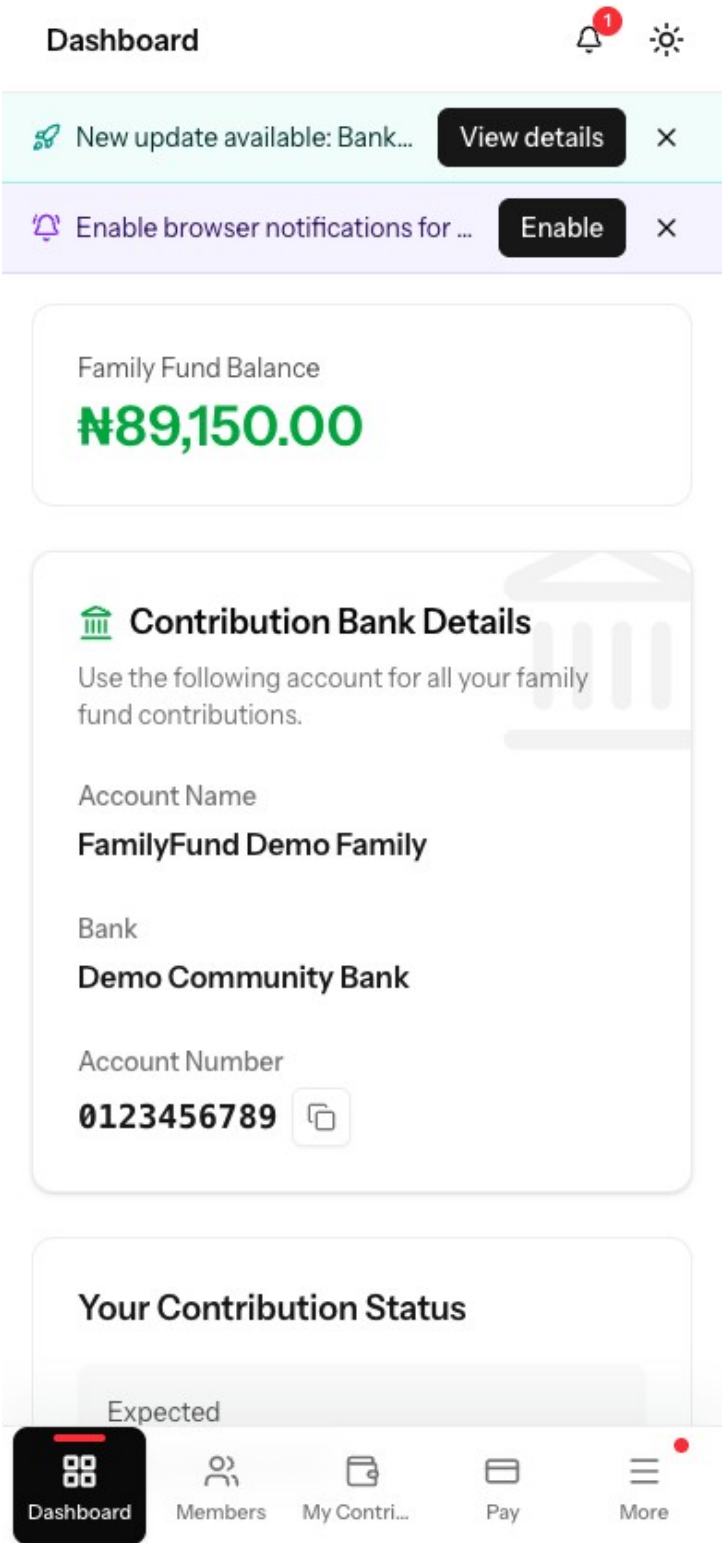
The screenshot displays the FamilyFund Admin interface. On the left is a sidebar with navigation links: Dashboard, Members, My Contributions, Payments, Notifications, Expenses, Fund Adjustments, Pay, AI Assistant, Reconciliation, Reports, **Payment Risk** (highlighted), What's New, Family Admin, Family Settings, Invitations, Subscription, Privacy Policy, and Terms of Service. The main content area is titled 'Payment Risk' and includes a notification bar at the top with three items: 'Get contribution reminders on WhatsApp' (with a 'Verify number' button), 'New update available: Bank Reconciliation & Faster Family Operations' (with a 'View details' button), and 'Enable browser notifications for contribution reminders.' (with an 'Enable' button). Below the notification bar, the 'Payment Risk' section is titled 'OFFICER DECISION SUPPORT' and 'Payment Risk Insights'. It contains an advisory insight box stating: 'Advisory insight, not an automated decision. This is not a credit score. It does not send reminders, restrict access, change contribution terms, or automate any action. An authorized officer remains responsible for every review and follow-up.' Below this is a grey box labeled 'Insights unavailable' with a status of 'UNAVAILABLE', stating: 'The payment-risk model could not be verified safely. There is not enough mature payment history to produce responsible guidance for this period.' Underneath are four review status boxes: 'Contributions reviewed' (0), 'Priority review' (0), 'Routine review' (0), and 'Unavailable' (0). At the bottom, there are 'Review filters' (with 'Contribution period' set to 'August 2026' and 'Review band' set to 'All review bands') and an 'Evidence model' section stating 'No validated model is active for this period. Scores remain unavailable until the readiness gate is met.' A small circular icon with a red 'X' is visible in the bottom right corner of the interface.

Figure 4.13: Reconciliation Workspace and Seeded Settlement Candidate



Finally, Figure 4.14 shows the member dashboard at a 390-pixel viewport. Navigation, summary cards and contribution information remain contained within the mobile width, supporting the focused Chromium result without treating one viewport as a complete usability study.

Figure 4.14: Responsive Member Dashboard at 390 Pixels



4.3 Testing Strategy and Procedures

4.3.1 Test Plan

Testing is organised by risk. Unit tests isolate calculations and matching rules. Feature tests exercise routes, policies, validation, database changes, jobs, and service integration. Browser tests cover rendered user journeys and responsive behaviour. External integration tests fake or stub providers where deterministic automated proof is required; final live checks are reported separately because they depend on credentials, network access, and provider state. Security evidence includes cross-family denial, role denial, signature validation, signed-link expiry, immutable financial records, and confirmation-before-write behaviour. Performance evidence must measure a declared dataset and environment rather than relying on subjective impressions.

Developer-authored automated tests are the only tester evidence currently identified. No peer-testing session, end-user acceptance session, usability survey, or controlled comparative user study is claimed. If those activities occur later, the participants, task script, consent treatment, sample size, and results must be recorded before they are discussed.

The final verification sequence should minimise evidence drift. Focused machine-learning, Laravel, PostgreSQL and browser checks, the canonical local quality gate, and the redacted screenshot journeys have been completed against the post-change worktree. The next evidence stage is to commit the reviewed revision and obtain a fresh hosted gate. Performance probes, recovery exercises and live integrations must identify that same revision. If a later fix changes application behaviour, affected evidence must be repeated; an earlier successful screenshot or hosted run cannot certify a newer commit.

For each TC row, the final “Actual” field should describe the observed state rather than repeat the expected result. A suitable entry might name a returned status, created record, denied response, unchanged count, or rendered total, together with its test or capture reference. A failed case should remain in the table with the failure, corrective action, and rerun result. This structure makes the test reproducible and avoids the weak statement that a page “works.”

Table 4.4: Testing Strategy and Evidence Sources

Test level	Objective	Primary evidence source	Final evidence required	Current status
Unit	Verify isolated allocation, fee, enum, matching, and configuration behaviour	Pest and pytest unit specifications	Canonical local gate passed within 1,421 Pest tests/6,904 assertions at 100.0% PHP coverage; 66 Python tests passed at 100% coverage	Met locally
Feature	Verify authorised application workflows and persistence	Pest feature specifications	Canonical local gate passed; focused payment-risk suite also passed 138 tests/440 assertions at 100% payment-risk-file coverage	Met locally
Integration	Verify payments, reports, jobs, channels, AI tools, and reconciliation boundaries	Service/feature specifications with fakes and PostgreSQL smoke	Canonical integrations passed; disposable PostgreSQL migration and focused rerun passed 138 tests/440 assertions	Met locally; live providers remain separate
Browser/system	Verify complete responsive user journeys	Pest browser specifications and redacted running-system captures	Focused predictive browser run passed 5 tests/82 assertions; 20 synthetic-tenant images captured and inspected	Met for the recorded local journeys
Security	Verify tenant, role, signature, immutable-record, and signed-link controls	Policies and negative-path specifications	Canonical local negative paths passed; focused predictive authorisation, tenant, immutability, and advisory-only paths also passed	Met for automated scope
External/live	Verify Paystack, message delivery, hosted services, and AI provider	Authorised test accounts and hosted environment	Time-stamped final observations	Not Executed

Test level	Objective	Primary evidence source	Final evidence required	Current status
Predictive	Verify data contract, baselines, logistic regression, metrics, artefact, and activation	PRED-01–PRED-10	Software verification passed in Python, Laravel, PostgreSQL and Chromium; authorised CSV and evaluation artefacts absent	Partially Met
Performance	Quantify response time and resource use against NFRs	Declared benchmark dataset/environment	Raw results, summary table, and chart	Not Executed

4.3.2 Structured System Test Cases

The following cases cover the functional requirements and the major post-design controls. “Met” means the relevant deterministic checks passed in the fresh canonical local run. “Partially Met” means the automated portion passed but the case also requires external-provider, hosted, rendered, recovery, or performance evidence that was not executed. It does not convert an absent live observation into a pass.

Table 4.5: Structured System Test Cases and Current Results

ID	Test objective	Preconditions	Test input(s)	Expected output	Actual output/evidence	Status
TC-01	Authenticate and route a verified user	Registered verified account	Valid and invalid credentials	Valid login reaches current family; invalid login is rejected	Authentication, registration, reset, and email-verification cases passed in the canonical 1,421-test run; Figures 4.1a–4.1c record the interface boundary	Met
TC-02	Enforce 2FA and passkey ownership	Authenticated account with security setup	Valid challenge, invalid challenge, another user's passkey	Valid challenge succeeds; invalid/foreign credential fails	Passkey, 2FA, challenge, and allowed-origin cases passed in the canonical run	Met
TC-03	Create and switch isolated family workspaces	User with two memberships	Family slug/current-family switch	Context changes only to a joined family	Membership-routing, family-switching, and predictive tenant-switch browser cases passed	Met
TC-04	Invite a member and assign permitted role/category	Family administrator; active category	Valid, expired, duplicate, and invalid-role invitations	Only valid invitation creates membership with permitted role/category	Invitation and role/category cases passed; external message delivery was not exercised	Partially Met
TC-05	Prevent cross-family and ordinary-member administration	Two families and roles	Foreign record URL and restricted form submission	Request is denied without data exposure or mutation	Role, policy, membership, report, AI-isolation, and predictive member-denial cases passed	Met
TC-06	Generate monthly obligations without duplicates	Active paying memberships	Month generation repeated for same period	One obligation per eligible membership/period	Generation, category-history, and idempotency cases passed in the canonical run	Met

ID	Test objective	Preconditions	Test input(s)	Expected output	Actual output/evidence	Status
TC-07	Calculate contribution status at boundaries	Paid, partial, unpaid, due, and overdue records	Amounts around balance and dates around due date	Correct status and balance are shown	Due-date, status, partial-payment, and dashboard cases passed in the canonical run	Met
TC-08	Allocate partial and lump-sum payments oldest first	Member with several incomplete months	Partial amount and multi-month amount	Oldest balances receive money first; totals equal receipt	Allocation, balance-first, and partial-payment cases passed in the canonical run	Met
TC-09	Enforce advance-payment and invalid-amount limits	Member with current obligations	Zero, excessive, and within-limit future amounts	Invalid amounts fail; permitted amount allocates within limit	Advance-limit and payment-allocation boundary cases passed in the canonical run	Met
TC-10	Restrict manual payment recording by role	Admin, secretary, and member accounts	Same valid payment request from each role	Admin/secretary permitted; member denied	Payment policy, request validation, and recording cases passed in the canonical run	Met
TC-11	Verify Paystack before applying contribution	Known initiated reference	Valid, invalid-signature, unknown, duplicate, and mismatched events	Only valid known matching event settles once	Deterministic service, webhook, ledger, fee, mismatch, and duplicate-processing cases passed; live gateway not executed	Partially Met
TC-12	Record and reverse expenses/adjustments immutably	Authorised officer and existing record	Valid create, direct update, correction/reversal	Create succeeds; direct mutation denied; reversal remains auditable	CRUD, policy, observer, reversal, and governed AI-tool cases passed in the canonical run	Met
TC-13	Generate consistent monthly/annual reports	Family with financial records	Same filters for screen, CSV, PDF, scheduled output	Formats use identical shared totals and private access rules	Monthly, annual, Phase Three, export, private-artifact, and schedule cases passed; Figures 4.8a–4.8b record the controls and generated output	Met for local evidence

ID	Test objective	Preconditions	Test input(s)	Expected output	Actual output/evidence	Status
TC-14	Limit member dashboard/history to authorised records	Member and officer accounts	Own and another member's history	Member sees own records; officer access follows role	Dashboard, member-history, contribution, and payment-policy cases passed in the canonical run	Met
TC-15	Send reminders only through eligible channels	Overdue contribution and configured/unconfigured channels	Email, push, WhatsApp, invalid selection	Valid eligible selection sends; unavailable choice is reported	Reminder command, channel, notification, webhook, and MCP cases passed with fakes; external delivery not executed	Partially Met
TC-16	Enforce subscription features and member limits	Families on different plans	Feature request and add-member at/under limit	Plan rules allow or reject consistently	Subscription, strict Payment Risk entitlement, and plan-limit cases passed in the canonical run	Met
TC-17	Keep AI reads tenant- and role-scoped	Configured test user and family data	Balance, contribution, expense, and member queries	Tools use only permitted family data	AI tool, privacy, family-context, and role cases passed with deterministic fakes; live provider not executed	Partially Met
TC-18	Require confirmation and permission for AI writes	Officer/member accounts	Preview, confirmed repeat, unauthorised write	Preview does not mutate; exact confirmed authorised request may execute	AI preview, confirmation, idempotency, role, payment, expense, adjustment, generation, and invitation cases passed; live provider not executed	Partially Met
TC-19	Reconcile only unique exact matches automatically	Imported credit with unique, ambiguous, and absent references	Match request for each case	Unique exact candidate links; ambiguity remains suggested/unlinked	Matching-service, Phase Four, and reconciliation-browser cases passed in the canonical run	Met

ID	Test objective	Preconditions	Test input(s)	Expected output	Actual output/evidence	Status
TC-20	Guard closed reconciliation periods and provider settlements	Open/closed periods and settlement records	Mutation, close, reopen, settlement match	Closed period blocks mutation; authorised reopen is auditable	Period, mutation, reopening, provider-settlement, and browser cases passed in the canonical run	Met
TC-21	Archive, retain, restore, and purge under policy	Active and archived family/member records	Archive, restore, legal hold, expired retention	Lifecycle follows retention and legal-hold rules	Archive, restore, retention, purge, legal-hold, and backup-configuration cases passed; hosted recovery drill not executed	Partially Met
TC-22	Demonstrate responsive hosted operation	Final deployed revision with populated test family	Mobile/desktop journeys, queues, scheduler, backup, /up and performance probes	Pages are readable, services healthy, and declared thresholds met	Focused predictive Chromium run passed 5 tests/82 assertions, including 390-pixel and dark-mode checks; hosted SHA, report screenshots, service checks, and measurements are unavailable	Partially Met

4.3.3 Prototype Tests and Deferred Predictive Evaluation

The predictive cases are separated because application tests cannot substitute for a real training dataset and held-out evaluation. Each case must retain its raw configuration or machine-readable output in the final evidence package.

Table 4.6: Predictive Prototype Test and Evidence Matrix

ID	Test objective	Preconditions	Input(s)	Expected output	Actual output/evidence	Status
PRED-01	Validate authorised dataset presence and provenance	Approved private export	CSV path and provenance record	Readable dataset with documented owner, period, and privacy treatment	No authorised predictive-training CSV is present	Not Executed
PRED-02	Validate schema, types, history, and both classes	PRED-01 complete	Contribution/payment fields and provenance	Invalid or insufficient data fails closed with reasons	Python quality-gate tests passed within the 66-test, 100%-coverage run; no private-data output	Partially Met
PRED-03	Build leakage-safe chronological partitions	PRED-02 complete	Dated labelled records	Training, validation, and untouched test periods with no future features	Point-in-time and complete-period 60/20/20 split tests passed; no private split manifest	Partially Met
PRED-04	Evaluate training-prevalence baseline	PRED-03 complete	Training prevalence and held-out labels	Accuracy, balanced accuracy, overdue precision/recall/F1, PR-AUC, ROC-AUC when defined, Brier score, support, prevalence, and confusion matrix	Baseline metric and serialization tests passed; no private held-out result	Partially Met
PRED-05	Evaluate previous-period-late baseline	PRED-03 complete	Prior outcome and held-out labels	The same complete metric set and confusion matrix on the identical held-out partition	Previous-period baseline tests passed on controlled fixtures; no private result	Partially Met
PRED-06	Fit approved logistic regression only	PRED-03 complete	Training features/labels and fixed configuration	Versioned coefficients, intercept, scaler and threshold	Deterministic L2 training, artefact serialization and Python/PHP golden-score parity passed; no fitted private artefact	Partially Met

ID	Test objective	Preconditions	Input(s)	Expected output	Actual output/evidence	Status
PRED-07	Select and evaluate without test leakage	PRED-04–06 complete	Validation selection then one held-out evaluation	Full metric table, curves, class distribution, intervals, and confusion matrix	Threshold, held-out protocol, bootstrap and chart tests passed within the 66-test run; no private evaluation artefacts	Partially Met
PRED-08	Record error and coefficient analysis	PRED-07 complete	False positives/negatives and fitted coefficients	Interpretable limitations without causal overclaiming	Aggregate chart and coefficient-factor tests passed; no private error analysis	Partially Met
PRED-09	Activate only a passing versioned artefact	PRED-07 complete and gates satisfied	Model metadata and inference request	Authorised result or explicit unavailable reason; no automatic sanction	Laravel validation, checksum, immutability, activation, idempotency and advisory-only tests passed within 138 tests/440 assertions; no active artefact	Partially Met
PRED-10	Capture populated UI and performance evidence	PRED-09 complete	Sufficient-history and cold-start members	Captioned result/unavailable screenshots and timed inference	Focused Chromium run passed 5 tests/82 assertions for synthetic populated and unavailable states; real-model screenshot and performance results absent	Partially Met

4.3.4 Performance, Security, and Usability Procedure

Final performance measurement must declare the hardware, hosted revision, database, record counts, warm-up method, number of repetitions, and percentile reported. The prescribed benchmark uses synthetic families of 25, 75 and 250 members over 12 and 24 months. After a documented warm-up, each case is repeated three times. Evidence records batch duration, per-score median and p95, page and refresh p50/p95, database-query count, peak memory, private artefact size, and 30-second concurrency probes. The operational acceptance thresholds are a page and refresh p95 below two seconds for the 75-member case, a 250-member scoring batch below 30 seconds, bounded query growth without an N+1 pattern, and zero request errors. These are declared operational thresholds, not retrospectively attributed to the original Chapter Three NFRs.

The benchmark dataset must reflect ordinary family sizes without containing private production records. Its construction, member count, contribution periods, receipt allocations, expenses, adjustments, bank transactions, and report rows should be recorded. Cold and warm measurements should not be mixed. Query-cache, application-cache, and browser-cache conditions need to be stated, and enough repetitions should be retained to show a distribution rather than one unusually fast response. Predictive inference timing is meaningful only after an activated artifact exists; a controller returning “model unavailable” measures the guard path, not model performance.

Security validation should combine automated negative paths with configuration review. Tenant and role denials, signed URLs, Paystack signatures, AI confirmation, predictive authorisation, immutable records, secret handling, HTTPS, backup configuration, queue/scheduler operation, and basic dependency/security checks are in scope. Usability evidence is limited to responsive browser workflows unless real participants are recruited. No satisfaction percentage or Likert result may be added without an actual instrument and sample.

Basic vulnerability checking should be reported with its scope and date rather than as a claim that the system is “secure.” Dependency advisories, insecure transport, exposed secrets, missing authorisation, mass assignment, cross-family identifiers, forged webhooks, expired links, and direct mutation routes are relevant checks. Backup configuration alone is not recovery evidence: a final operational claim requires a controlled restore or equivalent

provider proof using non-production material. Where a live check cannot be performed safely, the limitation should remain visible.

4.4 Test Results and Discussion

4.4.1 Current Results and Non-Functional Evaluation

The current evidence supports a cautious but stronger conclusion. The repository contains implementation and tests for the core family-fund functions, including later reporting, retention, reconciliation, and browser workflows. On 15 August 2026, the canonical local gate passed 1,421 Pest tests with 6,904 assertions and 100.0% PHP coverage in 22.209 seconds. PHPStan, Pint, Prettier, ESLint, and both client and server-side-rendering builds also passed. The locked predictive package passed 66 Python tests with 100% statement coverage. Focused evidence further recorded 138 Laravel payment-risk tests with 440 assertions and 100% coverage across the 16 payment-risk PHP files, the same 138 tests and 440 assertions on PostgreSQL, five Chromium tests with 82 assertions, and 171 combined Laravel tests with 517 assertions. Those focused runs overlap the canonical gate and are reported separately. The twenty inspected running-system images provide additional local interface evidence. No post-change hosted SHA is claimed; the older hosted baseline a699a821 predates this work and must not be substituted.

Table 4.7: Non-Functional Requirement Evaluation

Requirement	Evidence currently available	Required final result	Status
NFR1 Tenant isolation	Canonical family-routing, policy, report, AI-isolation, and predictive tenant-switch checks passed	Fresh negative-path suite with no cross-family exposure	Met for automated scope
NFR2 Role restriction	Canonical role, policy, request, and predictive officer/member checks passed	Fresh suite and browser proof for each role	Met for automated scope
NFR3 Strong authentication	Canonical Fortify, verification, 2FA, passkey, and browser checks passed	Fresh feature and browser proof	Met for automated scope

Requirement	Evidence currently available	Required final result	Status
NFR4 Deterministic allocation	Canonical oldest-first, partial, lump-sum, balance, and boundary checks passed	Fresh allocation suite including boundary cases	Met
NFR5 Paystack idempotency	Reference/signature/settlement logic and webhook tests	Fresh suite plus authorised live test transaction	Partially Met
NFR6 Responsive usability	Focused Chromium run passed mobile, dark-mode, keyboard, refresh and error checks for predictive states	Complete report screenshot set and wider journey review	Partially Met
NFR7 Understandable reports	Shared report service, labels, CSV/PDF/receipt generation	Rendered final report inspection and, if conducted, user feedback	Partially Met
NFR8 Performance	No final measurements	Declared benchmark dataset, response percentiles, and thresholds	Not Executed
NFR9 Maintainability	Separate services, actions, policies, jobs, and components; canonical static analysis, formatting, lint, builds, and 100% PHP/Python coverage passed	Clean canonical full gate	Met locally
NFR10 Auditability	Canonical immutable-batch, reversal, artifact, reconciliation-link, and period checks passed	Fresh audit/reconciliation tests and rendered trail	Met for automated scope; report capture pending
NFR11 AI privacy	Family-scoped tools and role-aware source/tests	Fresh tests plus authorised live conversation review	Partially Met
NFR12 Availability	Deployment, backup, queue, scheduler configuration/tests	Final hosted health, worker, scheduler, and recovery evidence	Not Executed
NFR13 Model governance	Python 66-test/100%-coverage run and Laravel 138-test/440-assertion run verified provenance/schema validation, chronological training, baselines, checksummed artefact and activation gates	Versioned private dataset, executed split, model and evaluation evidence	Partially Met

Requirement	Evidence currently available	Required final result	Status
NFR14 Model safety	Laravel, PostgreSQL and Chromium runs verified history tiers, immutable advisory records, unavailable/pooled paths, role denial and no action side effects	Authorised active-model inference and real populated screenshot	Partially Met

4.4.2 Deferred Predictive Training and Evaluation

Predictive training and empirical evaluation were Not Executed. This is an incomplete objective, not a positive model finding. Without authorised historical records, the study could not establish baseline performance, fitted logistic-regression coefficients or held-out accuracy. Synthetic fixtures can verify that training code runs and scoring calculations agree, but they cannot show how accurately the proposed model would predict real family payment behaviour. Predictive validation is therefore assigned to future work.

Table 4.8: Predictive Training and Evaluation Evidence Status

Evidence item	Required evidence	Actual result	Status
Dataset and provenance	Dataset identifier, owner/consent basis, SHA-256 digest, time window, row/member/period counts and class distribution	No authorised private CSV or provenance file supplied	Not Executed
Chronological split	Training, validation and held-out periods, rows and class counts	No private split manifest produced	Not Executed
Logistic regression	Configuration, threshold, coefficients, intercept and held-out metrics with intervals	No fitted private artefact or metrics produced	Not Executed
Training-prevalence baseline	Same held-out metrics and confusion matrix	No private baseline result produced	Not Executed
Previous-period-late baseline	Same held-out metrics and confusion matrix	No private baseline result produced	Not Executed
Calibration, discrimination and error analysis	Evaluation curves, confusion matrix, coefficient and error review	No populated figures or error analysis produced	Not Executed

Evidence item	Required evidence	Actual result	Status
Artefact and activation	Model/schema versions, checksum, evidence package and activation decision	No candidate or active artefact installed	Not Executed
Inference and interface	Authorised populated result, insufficient-history state and timing	No populated screenshot or performance measurement	Not Executed

Future evaluation would need the actual class distribution, baseline comparisons, evaluation curves, confusion matrix, calibration and coefficient evidence, and error analysis. These absent outputs are not numbered as figures in the present report. Both baselines would need to use the same untouched held-out period as logistic regression. If a future model does not pass the specified evaluation criteria, the extension should remain inactive. The current report retains the partial verdict for the original FR20 requirement.

4.4.3 Discussion of Results

The implementation and executed test evidence indicate that FamilyFunds addresses the central engineering gap identified in Chapter Two more directly than notebooks, spreadsheets, or messaging threads. The application formalises family workspaces and roles, separates obligations from receipts, applies one deterministic allocation rule, records spending and corrections, and produces reviewable outputs. The canonical gate and predictive-focused tenant, role, persistence, PostgreSQL, and browser checks provide fresh local evidence. Production-readiness conclusions still depend on the hosted, live-provider, screenshot, recovery, performance, and authorised-data evidence identified in this chapter.

The tenant and role design is consistent with RBAC and multi-tenant literature: permissions are adapted to family administrator, financial secretary, member, and platform roles rather than copied from a commercial bank. Payment handling also responds to a domain-specific gap left by payment gateways. Paystack can report that a transaction succeeded, but it does not decide which family obligation the money satisfies. The oldest-balance-first service keeps that decision inside the governed application and makes partial and lump-sum payments reproducible.

The reporting and reconciliation work demonstrates the value of iterative development. Chapter Three originally focused on reports, expenses, adjustments, and payment verification. Implementation later showed that an audit-friendly system also needs a

controlled comparison between internal records and external bank transactions. Reconciliation is therefore a defensible extension of the auditability objective. Its narrow exact-match rule avoids treating an approximate suggestion as an authoritative financial link, which is particularly important where several receipts may share an amount or nearby date.

The controlled AI design addresses the fourth research question only partially. Static tests and source structure show role-aware tools, family-scoped queries, and confirmation before writes. They do not establish that a configured live provider will always produce accurate, understandable language or acceptable latency. This limitation is consistent with literature warning that fluent financial text may still contain unsupported claims. Final evaluation requires authorised provider-backed examples grounded in known report data, followed by comparison of the response with the source totals. Until that occurs, live AI relevance and usefulness remain Not Executed.

The revised fifth research question is addressed only at the level of data requirements and evaluation design. The original question about predictive performance remains unanswered. Logistic regression and the two baselines were selected, but no authorised historical dataset was obtained for their empirical comparison. Sparse histories, class imbalance and changing contribution expectations could all affect a future result. Its evaluation would therefore need overdue-class recall, precision, F1, PR-AUC, error analysis and comparison with both baselines, alongside accuracy. No predictive benefit follows from the software test results reported here.

The sixth research question is partially addressed by the clean canonical gate and the predictive-focused Python, Laravel, PostgreSQL and Chromium checks covering normal, boundary, invalid, authorisation, idempotency, immutability, and browser scenarios. Populated report screenshots, the performance run, hosted services, live integrations, and recovery evidence still prevent a complete operational claim. Honest status reporting remains preferable to importing historical counts from an earlier revision because the evidential question is whether the integrated system passes at each claimed boundary.

4.4.4 Objectives and Research Questions Verdicts

Table 4.9: Objectives and Research Questions Verdict Matrix

Item	Current evidence	Verdict
Objective 1: Review related practice and literature	Chapter Two concepts, empirical works, existing systems, and gap	Met for report scope
Objective 2: Design the multi-tenant system	Chapter Three requirements, architecture, database, algorithms, and diagrams	Met for design scope
Objective 3: Implement the proposed system	Broad current source, clean canonical local gate, and predictive-focused Laravel, PostgreSQL and browser evidence	Partially Met overall; local implementation gate passed, hosted evidence pending
Objective 4: Integrate controlled AI	Role-aware agents/tools and static tests	Partially Met; live provider evidence pending
Objective 5: Explore a future predictive extension	Prototype and data/evaluation requirements documented; original empirical training and comparison absent	Partially Met against the original objective; evaluation Not Executed
Objective 6: Test key requirements	TC evidence mapping plus clean canonical, Python, PostgreSQL and Chromium checks and twenty inspected interface images	Partially Met; external, recovery, and performance evidence pending
RQ1: Required design features	Chapters Two and Three plus implementation traceability	Answered at design level
RQ2: Tenant and role protection	Layered implementation plus canonical and predictive route, tenant-switching and member-denial checks	Validated in the local automated scope; hosted review pending
RQ3: Consistent payment allocation	Transactional oldest-first implementation and canonical normal/boundary checks	Validated in the local automated scope
RQ4: Controlled AI usefulness	Governance design and static tests, no live output	Partially answered

Item	Current evidence	Verdict
RQ5: Requirements for future predictive evaluation	Data contract and validation protocol documented; no empirical model evidence	Design requirements specified; original performance question unanswered
RQ6: Functional, security, reliability, and usability testing	Broad specification set plus fresh focused local and PostgreSQL/browser results; external and performance evidence absent	Partially answered

4.4.5 Summary of the Chapter

This chapter connected the Chapter Three design to identifiable FamilyFunds modules, code listings, and test specifications. The application contains substantial static evidence for tenant-aware membership, role control, contribution generation, deterministic payment allocation, Paystack handling, expenses, adjustments, reports, reminders, controlled AI tools, retention, and the later reconciliation extension. It also defined twenty-two system cases and ten predictive evidence gates.

The chapter did not convert pending work into claimed success. The canonical local application gate passed 1,421 tests, 6,904 assertions and 100.0% PHP coverage, while 66 Python tests reached 100% statement coverage. Focused Laravel, PostgreSQL and Chromium checks also passed, and twenty redacted synthetic-tenant images provide running-system evidence for the recorded journeys. Hosted checks, performance measurements, recovery proof, live-provider acceptance, and authorised-data predictive training remain pending or Not Executed. No private predictive CSV was available, so logistic-regression and baseline metrics were not invented and the feature cannot be treated as activated. The conventional application is locally verified, but operational readiness and the predictive objective remain partially met at the current evidence boundary; predictive evaluation itself is Not Executed. Chapter Five must preserve these verdicts unless newer, reproducible evidence replaces them.

CHAPTER FIVE

SUMMARY, CONCLUSION AND RECOMMENDATIONS

5.1 Introduction

This chapter brings together the main findings of the study and considers what they mean for the original aim of designing and implementing FamilyFunds. It draws its conclusions from the requirements, design, implementation evidence and test results presented in the preceding chapters. It does not introduce a new experiment or convert an unexecuted activity into a successful result. In particular, the conclusions about predictive analytics remain bounded by the absence of an authorised historical training dataset, while the conclusions about hosted operation and live external services remain bounded by the evidence gaps reported in Chapter Four.

The discussion is organised around the research findings, the extent to which each objective was achieved, the study's contributions and its practical limitations. It then presents recommendations for completing the outstanding validation work and for operating the implemented system responsibly. The final sections identify suitable directions for later research and disclose the role of artificial intelligence tools in supporting the project work.

5.2 Summary of Findings

The study began with a practical problem: informal family contribution funds are often managed through notebooks, spreadsheets and messaging threads that do not provide one consistent account of obligations, payments, expenses and balances. The literature and system review showed that existing tools tend to address only part of this problem. General payment applications move money but do not apply family contribution rules; spreadsheets are flexible but weak in access control and audit history; and cooperative platforms are usually designed for a more formal organisational setting. This supported the need for a purpose-built system that combines family-level tenancy, defined roles, deterministic payment allocation, reporting and controlled automation.

The design findings established that logical data separation cannot depend on a single interface choice or hidden identifier. FamilyFunds therefore combines the active family context, membership records, family-scoped queries, validation and policies. Roles are

attached to a person's membership in a particular family rather than treated only as a global user property. This allows the same person to hold different responsibilities in different family groups without automatically carrying privileges from one context into another. The local automated evidence reported in Chapter Four supports this design at the tested boundary, although final hosted review is still required before making a wider operational claim.

The financial implementation also confirmed the value of separating an obligation from the receipt used to settle it. Monthly contributions record what a member is expected to pay, while payment batches and allocation lines record what was received and how it was applied. The oldest-balance-first rule gives partial and lump-sum payments a repeatable outcome: earlier outstanding periods are settled before later ones, subject to the defined advance-payment limit. Transaction boundaries, row locking, idempotency controls and explicit reversal records protect the ledger from duplicate or conflicting operations. This directly answers the third research question within the local automated scope.

The broader application integrates contribution generation, manual and Paystack-related payment handling, expenses, fund adjustments, reminders, reports, receipts, scheduled outputs, account security and subscription controls. Reconciliation emerged during implementation as a useful post-design extension. It provides a controlled means of comparing internal records with imported bank transactions without pretending that this feature belonged to the original functional baseline. This distinction is important because it presents the development history as it occurred and shows how iterative software work can reveal an audit requirement that was not fully visible at the design stage.

The controlled AI work produced a governed implementation rather than an unrestricted financial chatbot. Tools obtain current family data through the application, their availability depends on the user's role, and write actions require a preview followed by explicit confirmation. These controls reduce the risk that generated language will bypass established permissions or silently alter financial records. However, the study did not execute the planned live provider acceptance matrix. The evidence therefore supports the architecture and automated safeguards, but it does not establish the factual quality, clarity or latency of live model responses. The fourth research question is consequently only partially answered.

The exploratory predictive work produced a software prototype and a protocol for future evaluation. It includes chronological splitting, leakage checks, logistic-regression code,

baseline methods and controls for installing and using a model. Tests exercised these mechanisms using controlled fixtures. However, no authorised historical dataset was obtained, so no model was trained or activated for practical use and no accuracy, F1, AUC or calibration result is claimed. The revised fifth research question identifies the required data and validation procedure; the original question about predictive performance remains unanswered. This work is retained as a foundation for future research rather than a demonstrated benefit of FamilyFunds.

Testing provided strong local evidence while also revealing the difference between implementation quality and complete operational validation. The canonical local gate passed 1,421 Pest tests with 6,904 assertions and 100.0% PHP coverage, alongside the static-analysis, formatting, lint and client and server-side-rendering build checks reported in Chapter Four. The locked predictive package passed 66 Python tests with 100% statement coverage, and focused Laravel, PostgreSQL and Chromium runs exercised the payment-risk boundary. Twenty redacted images from the synthetic development tenant also document the principal running-system journeys. These results support the conventional application and predictive software implementation locally. They do not replace the pending hosted run, live Paystack and messaging evidence, live AI review, recovery exercise, performance measurements or private-data model evaluation. The sixth research question is therefore partially answered rather than fully closed.

5.3 Achievement of Objectives

Table 5.1: Achievement of Study Objectives

Objective	Evidence-based outcome	Verdict
1. Review relevant practice, tools and literature	Chapters One and Two established the problem context, related concepts, existing systems and research gap.	Met for report scope
2. Design the multi-tenant family fund system	Chapter Three specified requirements, architecture, data design, algorithms, interfaces, security and ethical safeguards.	Met for design scope

Objective	Evidence-based outcome	Verdict
3. Implement the proposed system	Core and extended modules are present, and the canonical local implementation gate passed; final hosted evidence is pending.	Partially Met overall
4. Integrate controlled AI features	Role-aware, family-scoped tools and confirmation controls are implemented and tested statically; live provider evidence was not executed.	Partially Met
5. Explore a future predictive extension	Data requirements, prototype code and evaluation protocol were prepared; the originally intended empirical training and comparison were not executed.	Partially Met against the original objective; evaluation Not Executed
6. Test the system against key requirements	Broad local, Python, PostgreSQL and Chromium checks passed and the principal interface journeys were captured; external, performance and recovery evidence remains outstanding.	Partially Met

The first two objectives were achieved within their stated academic boundaries. The review identified a coherent gap rather than assuming that every digital payment tool is a family fund system. The design then translated that gap into requirements and a layered architecture covering tenancy, roles, obligations, receipts, allocations, expenses, adjustments, reporting, security and controlled intelligent features. Together, these chapters supplied the foundation against which implementation could be traced.

The remaining objectives require more qualified verdicts. The third objective progressed beyond a prototype because the application modules and local quality gate provide substantial implementation evidence. Nevertheless, a locally passing system is not the same as a fully evidenced hosted service. The fourth objective established credible governance for AI-assisted functions, but usefulness cannot be concluded from source code and mocked responses alone. The fifth objective delivered the predictive engineering framework but not the empirical model result named in the research question. The sixth objective produced

broad automated evidence but stopped short of all external and non-functional checks. These partial verdicts are not failures hidden by wording; they identify exactly what has been completed and what a later acceptance exercise must still demonstrate.

5.4 Contributions of the Study

The first contribution is a domain-specific interpretation of multi-tenant financial administration for informal family groups. FamilyFunds treats each family as an independent workspace while recognising that one user may participate in several families. This model is more suitable than assigning one permanent global family role to a person, and it shows how membership context can be combined with policies and scoped queries to represent social responsibilities without weakening data isolation.

The second contribution is the explicit modelling of obligations, receipts and allocations as related but different records. This supports partial payments, lump-sum payments, oldest-balance-first settlement, correction and traceable reporting without rewriting historical obligations. In a family setting, that clarity is valuable because disagreements often concern not only whether money was paid, but also which period it settled and how the current balance was derived.

The third contribution is the integration of operational modules within one governed workflow. Contribution generation, payment processing, expenses, adjustments, reminders, reports, account security, reconciliation and retention are not isolated demonstrations. They share family context, authorisation and financial records. The later reconciliation extension also illustrates a defensible Agile response to an implementation finding: external bank evidence can be compared with internal records while authoritative links remain controlled and auditable.

The fourth contribution is the integration of controlled AI assistance within the existing application permissions. Assistant tools use family-scoped records, and proposed writes require explicit confirmation. The automated tests support those controls, although live response quality was not established. The separate predictive prototype is documented as preparatory engineering work; it is not claimed as a validated prediction method or an improvement in payment behaviour.

Finally, the report contributes an evidence-oriented method for evaluating a student software project. Static implementation evidence, fresh executed evidence and external evidence are

reported separately. This makes the findings reproducible and reduces the temptation to treat the existence of code, an earlier continuous-integration run or a synthetic demonstration as proof of current real-world behaviour.

5.5 Conclusion

The study concludes that a multi-tenant web application can provide a clearer and more accountable way to administer informal family contribution funds than fragmented notebooks, spreadsheets and messaging conversations. FamilyFunds brings member roles, contribution expectations, payment allocation, expenditure, adjustments and reporting into a shared but family-isolated system. Its deterministic allocation rule and immutable financial history address common sources of uncertainty, while authentication, policies and tenant-aware validation strengthen access control.

The recorded local tests and synthetic-tenant screenshots support the implementation of the main software workflows. FamilyFunds is therefore described as an AI-enhanced multi-tenant family fund management system, with AI enhancement referring to its controlled assistant and report-summary integration. Hosted operation, live integrations, recovery and performance still require further verification, and live AI response quality was not evaluated. Predictive analytics is excluded from the demonstrated outcomes: the project produced a prototype, but training and validation on authorised historical records remain future work. This revised scope reflects the defence feedback without upgrading any incomplete objective to an achieved result.

5.6 Limitations

The predictive extension is limited by the absence of an authorised historical dataset. The project could test schema validation, feature reconstruction, training code and scoring calculations, but it could not establish predictive performance on genuine member-period records. The original empirical comparison in Research Question Five was not completed. This limitation does not prevent the main application from recording payments, calculating balances or producing financial reports.

A second limitation is the boundary of the final execution evidence. Although the redacted screenshot register was completed locally, the hosted continuous-integration revision, performance workload, concurrency test, backup recovery exercise and final live-provider checks were not executed. Local automated and interface success offers confidence in the

implementation but cannot prove infrastructure health, production configuration or end-to-end delivery through third-party services.

The live usefulness of the AI assistant and report summaries is also unresolved. Role restrictions and confirm-before-write behaviour were tested, but generated responses were not assessed against a live provider for factual correctness, ambiguity, prompt injection, privacy and latency. In addition, the study did not conduct a formal user study with several family groups, so conclusions about ease of use and long-term adoption are based on design and browser checks rather than longitudinal field evidence.

Finally, the system depends on internet access and external services such as Paystack and configured communication providers. Its current payment design is centred on the supported Nigerian context, and the project does not attempt full double-entry accounting, direct bank integration, cryptocurrency support or operation at the scale of large regulated cooperatives. These boundaries should be considered when generalising the findings.

5.7 Recommendations

First, the project team should complete the declared release-evidence gate before presenting the system as production-ready. This should include a fresh hosted commit with green test and lint workflows, a healthy deployed application, worker and scheduler checks, a controlled backup-and-restore exercise, and a repeat of the redacted browser evidence set against that same hosted revision. The planned 25-, 75- and 250-member performance workloads should also be run with recorded percentiles, query counts, memory use and error totals.

Second, predictive activation should remain disabled until a formally authorised, pseudonymised dataset passes the provenance, maturity, history and class-balance checks. Training and validation should follow the chronological protocol defined in Chapter Three, and the untouched held-out period should be compared with both transparent baselines. Activation should occur only if the declared F1, balanced-accuracy and Brier-score gates are satisfied. The evaluation package, checksum and model version should be retained for audit, while the private source data and deployable artefact remain outside the public report.

Third, the controlled AI functions should undergo a supervised live acceptance exercise before wider use. Test prompts should cover factual report questions, tenant and role boundaries, missing data, ambiguous requests, prompt injection, member write denial and

officer preview-and-confirm flows. Each generated answer should be checked against authoritative application records. Officers should continue to review proposed actions, and AI output should never replace Laravel authorisation, validation or the financial ledger.

Fourth, operating procedures should accompany the software. Family administrators and financial secretaries need concise guidance on contribution generation, corrections, reversals, reconciliation, report handling and privacy. Access should be reviewed when a member's responsibility changes, expired report links should remain short-lived, and retention and legal-hold rules should be applied consistently. Regular restore drills and reconciliation reviews would turn the implemented audit features into dependable organisational practice.

Fifth, any extension to new countries or larger associations should begin with a requirements and compliance review rather than a simple interface change. Payment gateways, currencies, identity practices, record-retention duties and cooperative regulations vary by jurisdiction. The tenant and policy model can provide a foundation, but financial and operational rules must be validated for the intended setting.

5.8 Future Work

Predictive analytics is a future research direction. A study would first need authorised historical records from consenting family groups that meet the specified eligibility requirements. The existing logistic-regression prototype could then be trained and compared with both simple baselines using the same chronological partitions. An existing authorised dataset could be used if suitable; waiting a fixed number of months after deployment does not itself guarantee adequate data. Later work could examine calibration over time, subgroup errors, changing payment patterns and retraining criteria. Any validated output should remain advisory and subject to officer review.

A second direction is a structured field study of usability and adoption. Administrators, financial secretaries and members could complete realistic tasks on mobile and desktop devices, followed by interviews or standard usability measures. This would provide evidence about whether report labels, reminder flows, receipts and AI explanations are understandable to the intended users. It could also guide multilingual support, accessibility improvements and a low-connectivity or progressive-web-application mode.

A third direction is deeper financial interoperability. Additional verified payment gateways, bank-statement formats and carefully governed bank-data integrations could reduce manual

reconciliation work. More formal groups may also benefit from an optional double-entry accounting layer and export links to established accounting systems. These additions should retain the present separation between external provider events and authoritative family-owned records.

Further work could also study platform-scale tenancy under sustained load. Larger benchmark populations, multiple concurrent families, queue back-pressure and disaster-recovery scenarios would show where caching, indexing, archival or infrastructure changes become necessary. Such testing would provide a sounder basis for moving from a family-focused deployment to broader community or welfare-group use.

5.9 AI-Assistance Disclosure

Artificial intelligence tools supported parts of this project by assisting with code development, test planning, document structure and language editing. Their output was treated as a draft or technical aid rather than independent evidence. The student remains responsible for reviewing the implementation, confirming that descriptions match the repository and executed results, checking every citation against its source, and ensuring that the submitted report reflects the work that was actually completed.

No citation, dataset, model metric, test outcome or operational result should be included merely because an AI tool proposed it. Where required evidence was unavailable, the report records the item as pending, partially met or not executed. This disclosure does not transfer authorship or accountability to the tool; it makes the assistance transparent and preserves the student's responsibility for the final academic submission.

REFERENCES

- Adegbite, O. O., & Machethe, C. L. (2020). The gender gap in financial inclusion: Evidence from Nigeria. *World Development*, 127, 104755.
<https://doi.org/10.1016/j.worlddev.2019.104755>
- Adeyemo, R., & Bamire, A. S. (2005). Savings and investment patterns of cooperative farmers in Southwestern Nigeria. *Journal of Social Sciences*, 11(3), 183–192.
<https://doi.org/10.1080/09718923.2005.11892512>
- Akanji, O. O. (2006). Microfinance as a strategy for poverty reduction. *Central Bank of Nigeria Economic and Financial Review*, 39(4), 111–134.
- Aker, J. C., & Mbiti, I. M. (2010). Mobile phones and economic development in Africa. *Journal of Economic Perspectives*, 24(3), 207–232.
<https://doi.org/10.1257/jep.24.3.207>
- Amazon Web Services. (2020). SaaS tenant isolation strategies: Isolating resources in a multi-tenant environment. <https://docs.aws.amazon.com/whitepapers/latest/saas-tenant-isolation-strategies/saas-tenant-isolation-strategies.html>
- Anderson, S., & Baland, J.-M. (2002). The economics of ROSCAs and intrahousehold resource allocation. *Quarterly Journal of Economics*, 117(3), 963–995.
<https://doi.org/10.1162/003355302760193931>
- Apt, N. A. (2002). Ageing and the changing role of the family and the community: An African perspective. *International Social Security Review*, 55(1), 39–47.
<https://doi.org/10.1111/1468-246X.00113>
- Ardener, S. (1964). The comparative study of rotating credit associations. *Journal of the Royal Anthropological Institute of Great Britain and Ireland*, 94(2), 201–229.
<https://doi.org/10.2307/2844382>
- Ardener, S., & Burman, S. (Eds.). (1995). *Money-go-rounds: The importance of rotating savings and credit associations for women*. Berg Publishers.
- Aryeetey, E. (2008). From informal finance to formal finance in Sub-Saharan Africa: Lessons from linkage efforts. In *African Finance for the 21st Century* (pp. 1–24). International Monetary Fund.
- Bascom, W. R. (1952). The Esusu: A credit institution of the Yoruba. *Journal of the Royal Anthropological Institute of Great Britain and Ireland*, 82(1), 63–69.
<https://doi.org/10.2307/2844040>

- Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). Manifesto for Agile Software Development. <https://agilemanifesto.org>
- Bellotti, T., & Crook, J. (2009). Support vector machines for credit scoring and discovery of significant features. *Expert Systems with Applications*, 36(2), 3302–3308. <https://doi.org/10.1016/j.eswa.2008.01.005>
- Bezemer, C. P., & Zaidman, A. (2010). Multi-tenant SaaS applications: Maintenance dream or nightmare? In *Proceedings of the Joint ERCIM Workshop on Software Evolution (EVOL) and International Workshop on Principles of Software Evolution (IWPSE)* (pp. 88–92). ACM. <https://doi.org/10.1145/1862372.1862393>
- Bonneau, J., Herley, C., van Oorschot, P. C., & Stajano, F. (2012). The quest to replace passwords: A framework for comparative evaluation of web authentication schemes. In *Proceedings of the 2012 IEEE Symposium on Security and Privacy* (pp. 553–567). IEEE. <https://doi.org/10.1109/SP.2012.44>
- Bouman, F. J. A. (1995). Rotating and accumulating savings and credit associations: A development perspective. *World Development*, 23(3), 371–384. [https://doi.org/10.1016/0305-750X\(94\)00141-K](https://doi.org/10.1016/0305-750X(94)00141-K)
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32. <https://doi.org/10.1023/A:1010933404324>
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., ... & Amodei, D. (2020). Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, 33, 1877–1901.
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). ACM. <https://doi.org/10.1145/2939672.2939785>
- Chong, F., Carraro, G., & Wolter, R. (2006). Multi-tenant data architecture. Microsoft Developer Network (MSDN). [https://learn.microsoft.com/en-us/previous-versions/aa479086\(v=msdn.10\)](https://learn.microsoft.com/en-us/previous-versions/aa479086(v=msdn.10))
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
- Cowrywise. (2024). Cowrywise — Save and invest in dollars and naira. <https://cowrywise.com>
- CreditClan. (2024). CreditClan — Lending infrastructure for Africa. <https://creditclan.com>

- Dale, R. (2020). Natural language generation: The commercial state of the art in 2020. *Natural Language Engineering*, 26(4), 481–487.
<https://doi.org/10.1017/S1351324920000340>
- de Wynter, A., Wang, X., Sokolov, A., Gu, Q., & Chen, S.-Q. (2023). An evaluation on large language model outputs: Discourse and memorization. *Natural Language Processing Journal*, 4, 100024. <https://doi.org/10.1016/j.nlp.2023.100024>
- Demirgüç-Kunt, A., Klapper, L., Singer, D., & Ansar, S. (2022). The Global Findex Database 2021: Financial inclusion, digital payments, and resilience in the age of COVID-19. World Bank. <https://doi.org/10.1596/978-1-4648-1897-4>
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (pp. 4171–4186). ACL.
- EFInA. (2023). Access to Financial Services in Nigeria 2023 Survey. Enhancing Financial Innovation & Access. <https://efina.org.ng>
- Federal Republic of Nigeria. (2023). Nigeria Data Protection Act, 2023. <https://ndpc.gov.ng>
- Ferraiolo, D. F., Sandhu, R., Gavrila, S., Kuhn, D. R., & Chandramouli, R. (2001). Proposed NIST standard for role-based access control. *ACM Transactions on Information and Systems Security*, 4(3), 224–274. <https://doi.org/10.1145/501978.501980>
- FIDO Alliance. (2022). FIDO2: Web Authentication (WebAuthn).
<https://fidoalliance.org/fido2/fido2-web-authentication-webauthn/>
- Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures (Doctoral dissertation, University of California, Irvine).
- Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Gomber, P., Koch, J. A., & Siering, M. (2017). Digital finance and FinTech: Current research and future research directions. *Journal of Business Economics*, 87(5), 537–580.
<https://doi.org/10.1007/s11573-017-0852-x>
- Gugerty, M. K. (2007). You can't save alone: Commitment in rotating savings and credit associations in Kenya. *Economic Development and Cultural Change*, 55(2), 251–282.
<https://doi.org/10.1086/508716>

- Hand, D. J. (2009). Measuring classifier performance: A coherent alternative to the area under the ROC curve. *Machine Learning*, 77(1), 103–123.
<https://doi.org/10.1007/s10994-009-5119-5>
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer.
- Hu, V. C., Ferraiolo, D., Kuhn, R., Schnitzer, A., Sandlin, K., Miller, R., & Scarfone, K. (2014). *Guide to Attribute Based Access Control (ABAC) Definition and Considerations*. NIST Special Publication 800-162.
- Hussin Adam Khatir, A. A., & Bee, M. (2022). Machine learning models and data-balancing techniques for credit scoring: What is the best combination? *Risks*, 10(9), 169.
<https://doi.org/10.3390/risks10090169>
- Iganiga, B. O., & Asemota, A. (2008). The Nigerian unorganised rural financial institutions and operations: A framework for improved rural credit schemes in a fragile environment. *Journal of Social Sciences*, 17(1), 63–71.
- Jack, W., & Suri, T. (2014). Risk sharing and transactions costs: Evidence from Kenya's mobile money revolution. *American Economic Review*, 104(1), 183–223.
<https://doi.org/10.1257/aer.104.1.183>
- Jia, R., Yang, Y., Grundy, J., Keung, J., & Hao, L. (2021). A systematic review of scheduling approaches on multi-tenancy cloud platforms. *Information and Software Technology*, 132, 106478. <https://doi.org/10.1016/j.infsof.2020.106478>
- Kang, H., & Liu, X.-Y. (2023). Deficiency of large language models in finance: An empirical examination of hallucination (arXiv:2311.15548). *arXiv*.
<https://arxiv.org/abs/2311.15548>
- Karhapää, P., Behutiye, W., Rodríguez, P., Oivo, M., Costal, D., Franch, X., Aaramaa, S., Choraś, M., Partanen, J., & Abherve, A. (2021). Strategies to manage quality requirements in agile software development: A multiple case study. *Empirical Software Engineering*, 26, Article 28. <https://doi.org/10.1007/s10664-020-09903-x>
- Khandani, A. E., Kim, A. J., & Lo, A. W. (2010). Consumer credit-risk models via machine-learning algorithms. *Journal of Banking & Finance*, 34(11), 2767–2787.
<https://doi.org/10.1016/j.jbankfin.2010.06.001>
- Kouandou, A., & Zeh, I. P. (2024). Insuring consumption against shocks: The role of informal savings in Nigeria. *Food Policy*, 129, Article 102737.
<https://doi.org/10.1016/j.foodpol.2024.102737>

- Krasner, G. E., & Pope, S. T. (1988). A description of the model-view-controller user interface paradigm in the Smalltalk-80 system. *Journal of Object-Oriented Programming*, 1(3), 26–49.
- Lessmann, S., Baesens, B., Seow, H. V., & Thomas, L. C. (2015). Benchmarking state-of-the-art classification algorithms for credit scoring: An update of research. *European Journal of Operational Research*, 247(1), 124–136.
<https://doi.org/10.1016/j.ejor.2015.05.030>
- Li, W., Zhou, Y., Luo, S., & Dong, Y. (2022). Design factors to improve the consistency and sustainable user experience of responsive interface design. *Sustainability*, 14(15), 9131. <https://doi.org/10.3390/su14159131>
- Lyastani, S. G., Schilling, M., Neuber, M., Backes, M., & Bugiel, S. (2020). Is FIDO2 the kingslayer of user authentication? A comparative usability study of FIDO2 passwordless authentication. In *Proceedings of the 2020 IEEE Symposium on Security and Privacy* (pp. 268–285). IEEE.
- Masood, Z., Hoda, R., & Blincoe, K. (2020). How agile teams make self-assignment work: A grounded theory study. *Empirical Software Engineering*, 25, 4962–5005.
<https://doi.org/10.1007/s10664-020-09876-x>
- Mbiti, I., & Weil, D. N. (2016). Mobile banking: The impact of M-Pesa in Kenya. In S. Edwards, S. Johnson, & D. N. Weil (Eds.), *African Successes, Volume III: Modernization and Development* (pp. 247–293). University of Chicago Press.
<https://doi.org/10.7208/chicago/9780226315867.003.0009>
- Mikowski, M. S., & Powell, J. C. (2013). *Single page web applications: JavaScript end-to-end*. Manning Publications.
- Oloyede, J. A. (2008). Informal financial sector, savings mobilisation and rural development in Nigeria: Further evidence from Ekiti State of Nigeria. *African Economic and Business Review*, 6(1), 35–63.
- Ometov, A., Bezzateev, S., Mäkitalo, N., Andreev, S., Mikkonen, T., & Koucheryavy, Y. (2018). Multi-factor authentication: A survey. *Cryptography*, 2(1), 1.
<https://doi.org/10.3390/cryptography2010001>
- OpenAI. (2023). GPT-4 technical report. arXiv preprint arXiv:2303.08774.
- Osili, U. O. (2007). Remittances and savings from international migration: Theory and evidence using a matched sample. *Journal of Development Economics*, 83(2), 446–465. <https://doi.org/10.1016/j.jdeveco.2006.06.003>
- Otwell, T. (2024). *Laravel — The PHP framework for web artisans*. <https://laravel.com>

- Paystack. (2024). Paystack — Payments infrastructure for Africa. <https://paystack.com>
- PiggyVest. (2024). PiggyVest — Save and invest with confidence. <https://piggyvest.com>
- Reinink, J. (2024). Inertia.js — The modern monolith. <https://inertiajs.com>
- Reiter, E., & Dale, R. (2000). *Building Natural Language Generation Systems*. Cambridge University Press.
- Robisco, A. A., & Carbó Martínez, J. M. (2022). Measuring the model risk-adjusted performance of machine learning algorithms in credit default prediction. *Financial Innovation*, 8, Article 70. <https://doi.org/10.1186/s40854-022-00366-1>
- Sandhu, R. S., Coyne, E. J., Feinstein, H. L., & Youman, C. E. (1996). Role-based access control models. *IEEE Computer*, 29(2), 38–47. <https://doi.org/10.1109/2.485845>
- Schein, A. I., Popescul, A., Ungar, L. H., & Pennock, D. M. (2002). Methods and metrics for cold-start recommendations. In *Proceedings of the 25th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval* (pp. 253–260). ACM.
- Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game*. <https://scrumguides.org>
- Sharma, A., & Kaur, P. (2021). TOSDS: Tenant-centric object-based software defined storage for multitenant SaaS applications. *Arabian Journal for Science and Engineering*, 46(9), 9221–9235. <https://doi.org/10.1007/s13369-021-05743-z>
- Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson.
- Thomas, L. C. (2000). A survey of credit and behavioural scoring: Forecasting financial risk of lending to consumers. *International Journal of Forecasting*, 16(2), 149–172. [https://doi.org/10.1016/S0169-2070\(00\)00034-0](https://doi.org/10.1016/S0169-2070(00)00034-0)
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems*, 30, 5998–6008.
- W3C. (2021). *Web Authentication: An API for accessing Public Key Credentials Level 2*. World Wide Web Consortium. <https://www.w3.org/TR/webauthn-2/>
- Wezel, T., & Ree, J. J. (2023). Nigeria: Fostering financial inclusion through digital financial services (Selected Issues Paper No. 2023/020). International Monetary Fund. <https://doi.org/10.5089/9798400237195.018>
- You, E. (2024). Vue.js — The progressive JavaScript framework. <https://vuejs.org>